Software Design and Architecture

Fakhar Lodhi

Topics Covered: 076 – 082

Design Patterns – Part II

CamScanner

Software Design and Architecture

Topic#076
Façade Design Pattern

CamScanner

Problem

The application needs to be "object-orientized" to make it communicate/compatible with the rest of a newly built system.

How to go about it?

Facade Pattern

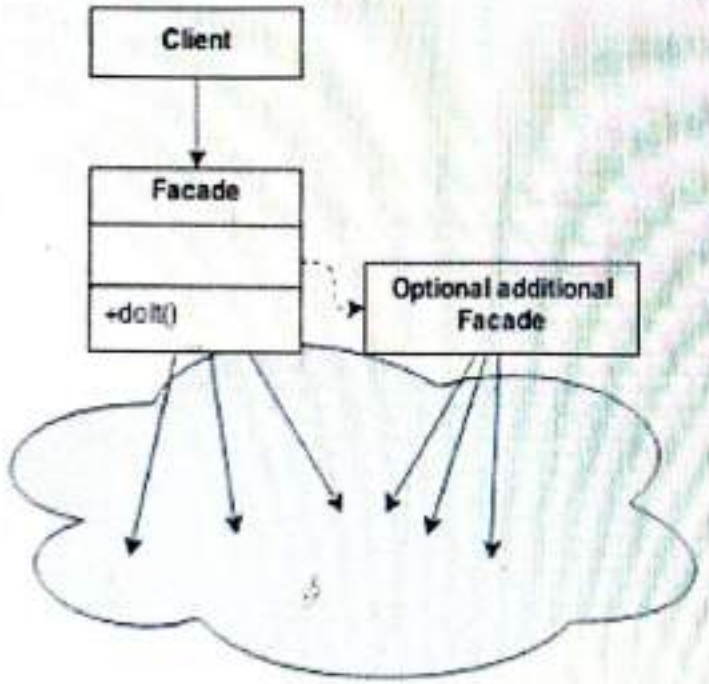
mcqs+ subjective

- **Structural Pattern**
- **Provides a unified interface to a set of interfaces in a subsystem.**
- **Defines a higher-level interface that makes the subsystem easier to use.**
- **Wraps a complicated subsystem with a simpler interface.**

CamScanner

Facade Pattern

- **Involves a single class which provides simplified methods required by client and delegates calls to methods of existing system classes.**
- **Can be used to add an interface to existing system to hide its complexities.**



Facade objects are often Singletons because only one Facade object is required.

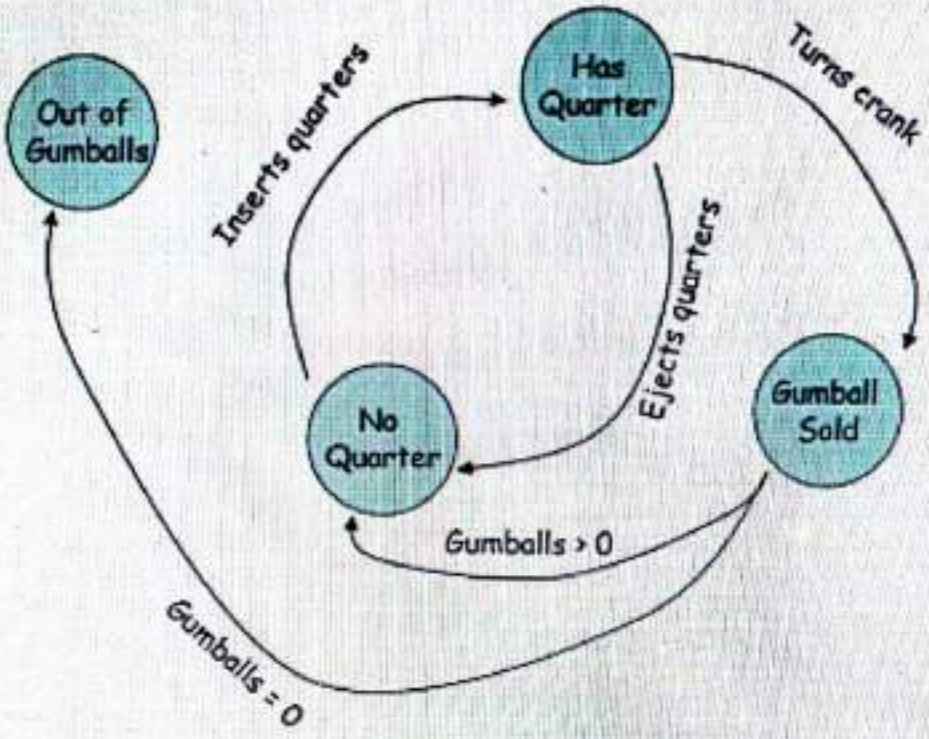
CamScanner

Software Design and Architecture

Topic#077
Modeling Sate Machine

CamScanner

Gumball Machine – Sate Diagram





Implementing State Machines

Step 1: Identify states

1. SOLD_OUT
2. NO_QUARTERS
3. HAS_QUARTERS
4. SOLD

CamScanner

Implementing State Machines
Step 2: Create instance variables to hold state

```
final static int SOLD_OUT        = 0;
final static int NO_QUARTERS     = 1;
final static int HAS_QUARTERS    = 2;
final static int SOLD            = 3;
```


Implementing State Machines
Step 3: Identify initial state

```
int state = SOLD_OUT;
int count = 0;
```

CamScanner

Implementing State Machines

Step 5: List all the actions that happen in the system

1. Inserts quarters
2. Turns crank
3. Dispense
4. Ejects Quarters

CamScanner

Software Design and Architecture

Topic#078
State Design Pattern

The State Pattern

The **State Pattern** allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

CamScanner

State Design Pattern: Motivation

- The State pattern is useful when you want to have an object represent the state of an application, and you want to change the state by changing that object.
- The State pattern is intended to provide a mechanism to allow an object to alter its behavior in response to internal state changes.

State Design Pattern: Motivation

- To the client, it appears as though the object has changed its class.
- The benefit of the State pattern is that state-specific logic is localized in classes that represent that state.

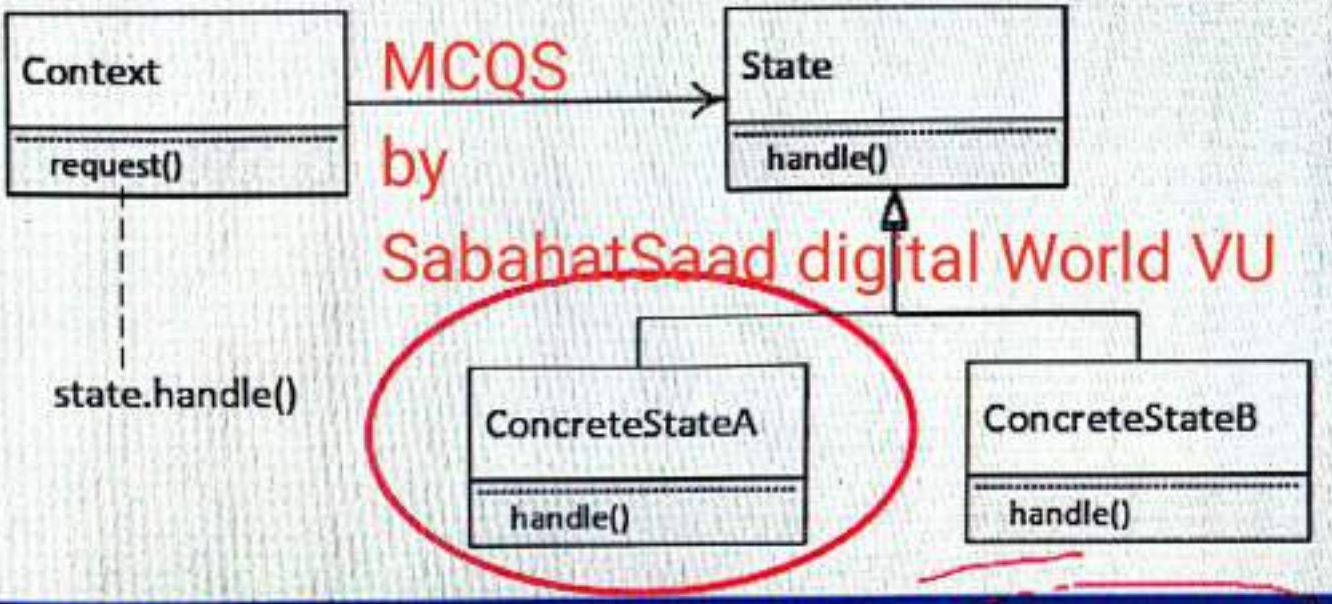
CamScanner

handle()

handle()

defines an interface for encapsulating the behavior associated with a particular state of the Context

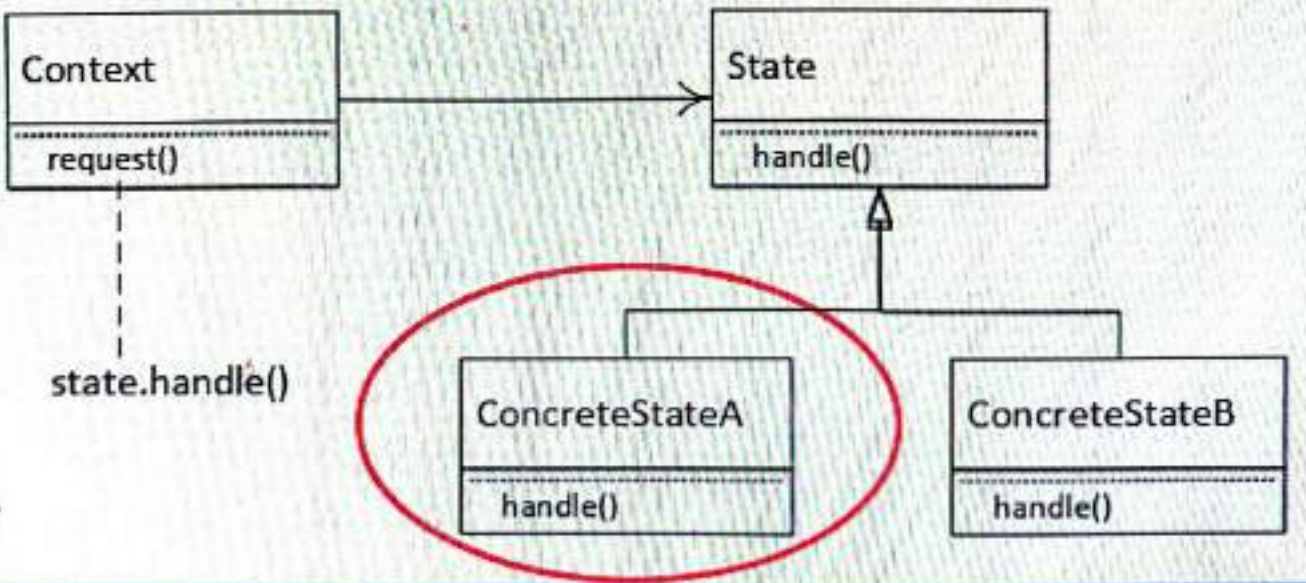
521



MCQS
by
SabahatSaad digital World VU

Each state is represented by a concrete state class – There are many concrete state classes

522



Concrete states handle request from the Context

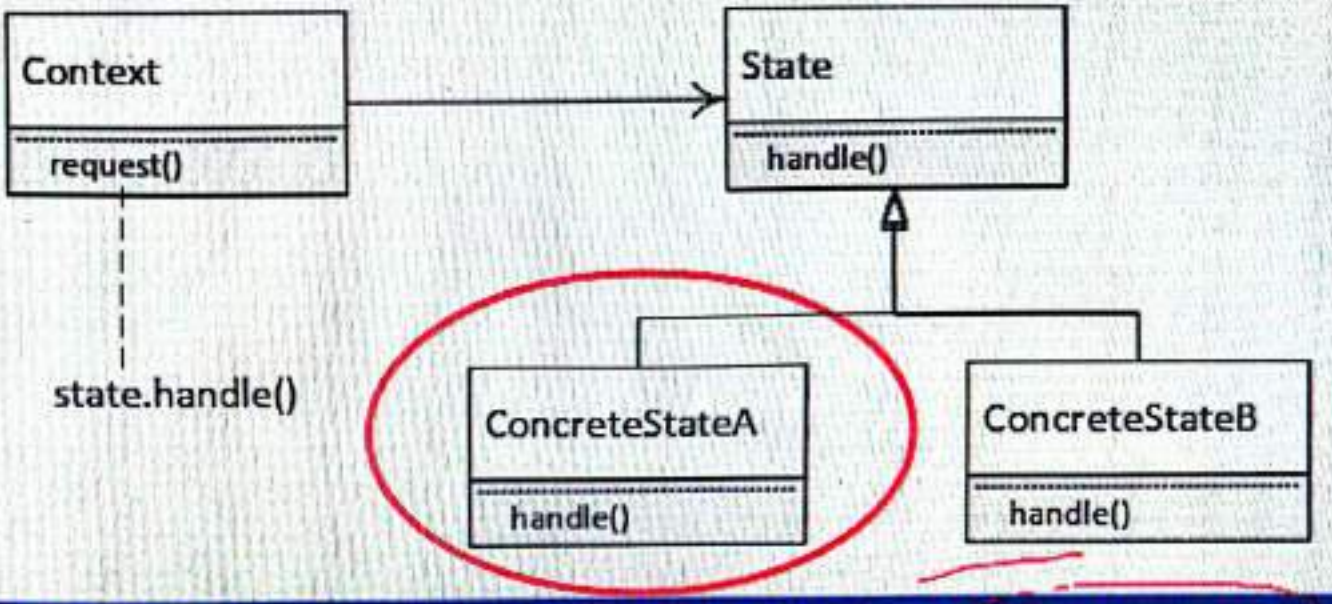
CamScanner

handle()

handle()

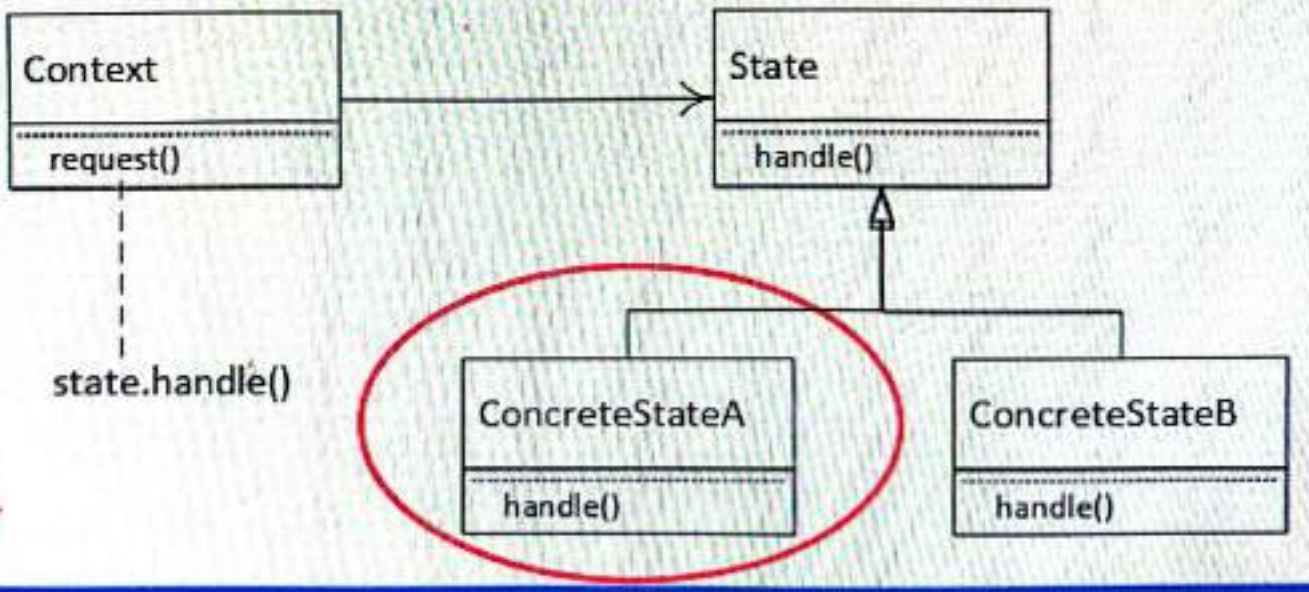
defines an interface for encapsulating the behavior associated with a particular state of the Context

521



Each state is represented by a concrete state class – There are many concrete state classes

522



Concrete states handle request from the Context

CamScanner

Gumball Machine - The new design

1. Define a State interface that contains a method for every action in the Gumball Machine
2. Implement a State class for every state of the machine. These classes will be responsible for the behavior of the machine when it is in the corresponding state
   - We are going to get rid of all the conditional code and instead delegate to the state class to do all the work

CamScanner

```
public class GumballMachine {
    ...
    public void insertQuarter() {
        state.insertQuarter();
    }
    ...
}
```

Just delegate.

That's it!!

- When an action is called, it is delegated to the current state
- It does the needful and sets the state to the next state

State Pattern - Summary

- Allows a context to change its behavior as the state of the context changes
- When an action is called, it is delegated to the current state
- It does the needful and sets the state to the next state
- By encapsulating each state in a class, we localize any changes that need to be ma[za]e.

CamScanner

State vs Strategy

- They are **both examples of association with delegation**
- The difference between the State and Strategy patterns is one of intent

BY Sabahat Saad
Digital World VU

State	Strategy
• Set of behaviors encapsulated in state objects; • At any time the context is delegating to one of those states. • Over time, the current state changes across the set of state objects to reflect the internal state of the context, so the context's behavior changes over time. Client knows very little, if anything, about the state objects • Alternative to putting a lots of conditional in the context	• Client usually specifies the strategy object that the context is associated with. • While the pattern provides the flexibility to change the strategy object at run time, there is one strategy object that is most appropriate for a context object. • Flexible alternative to subclassing: if you use inheritance to define the behavior of a class, you are stuck with it even if you need to change it. • With strategy you can change the behavior by associating it with different objects

CamScanner

- The difference between a tree data structure and the composite pattern

- In the case of a tree data structure, all nodes have the same structure and behavior

CamScanner

Observer Pattern

- Defines a "one-to-many" dependency between objects so that when one object changes state, all its dependents are notified and updated automatically

- a.k.a Dependence mechanism / publish-subscribe / broadcast / change-update

Subject & Observer

- Subject
  - the object which will frequently change its state and upon which other objects depend

- Observer
  - the object which depends on a subject and updates according to its subject's state.

CamScanner

Observer Pattern - Consequences

- Loosely Coupled
  - Reuse subjects without reusing their observers, and vice vers
  - Add observers without modifying the subject or other observers
- Abstract coupling between subject and observer
  - Concrete class of none of the observers is known
- Support for broadcast communication
  - Subject doesn't need to know its receivers

Software Design and Architecture

Topic#082

END!

CamScanner

Topic#083
Refactoring Introduction

What is Refactoring?

- Refactoring is the process of changing a software system such that:
  - the external behavior of the system does not change
  - the internal structure of the system is improved
  - This is sometimes called "Improving the design after it has been written"

CamScanner

Why is Refactoring Useful?

- The idea behind refactoring is to acknowledge that it will be difficult to get a design right the first time
  - as a program's requirements change, the design may need to change
  - refactoring provides techniques for evolving the design in small incremental steps

Refactoring Benefits

- Refactoring improves the design of software without refactoring, a design will "decay" as people make changes to a software system
- Refactoring makes software easier to understand because structure is improved, duplicated code is eliminated, etc.

CamScanner

Refactoring Benefits

- Refactoring helps you find bugs
- Refactoring promotes a deep understanding of the code at hand, and this understanding aids the programmer in finding bugs and anticipating potential bugs
- Refactoring helps you program faster because a good design enables progress

CamScanner

A (Very) Simple Example

Consolidate Duplicate Conditional Fragments

```
if (isSpecialDeal()) {
    total = price * 0.95;
    send ();
}
else {
    total = price * 0.98;
    send ();
}
```



```
if (isSpecialDeal())
    total = price * 0.95;
else
    total = price * 0.98;
send ();
```


A (Very) Simple Example

Better Still

```
if (isSpecialDeal())
    total = price * 0.95;
else
    total = price * 0.98;
send ();
```



```
if (isSpecialDeal())
    factor = 0.95;
else
    factor = 0.98;
total = price * factor;
send ();
```


CamScanner

Software Design and Architecture

Topic#085
Refactoring vs Rewriting and optimization

Refactoring versus Rewriting

- Code has to work mostly correctly before you refactor.
- If the current code just does not work, do not refactor.
  - Rewrite instead!

CamScanner

Optimization versus Refactoring

- The purpose of refactoring is
  - to make software easier to understand and modify
- Performance optimizations often involve making code harder to understand (but faster!)

Optimization versus Refactoring

```
for (i=0; i < N; i++)
{
    // condition does not
    // change inside the
    // loop

    if (cond1)        {//1...}
    else if (cond2)   {//2...}
    else              {//3...}
    // xyz
}
```



```
if (cond1) {
    for (i=0; i < N; i++) {
        //1...
        // xyz
    }
else if (cond2) {
    for (i=0; i < N; i++) {
        //2...
        // xyz
    }
else if (cond2) {
    for (i=0; i < N; i++) {
        //2...
        // xyz
    }
}
```


CamScanner

Topic#086
Making Refactoring Safe

How do you make refactoring safe?

1. use refactoring "patterns"

- Fowler's book assigns "names" to refactorings in the same way that the GoF's book assigned names to patterns

How do you make refactoring safe?

2. Test constantly!

- This ties into the extreme programming paradigm, you write tests before you write code, after you refactor code, you run the tests and make sure

CamScanner

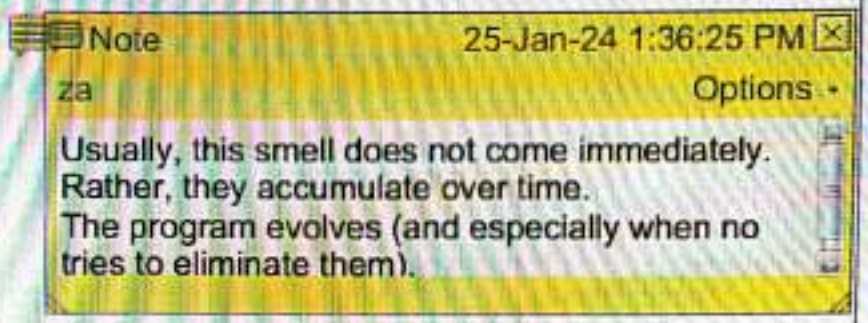
Bloaters (5)

- Bloaters are code, methods and classes that have increased to such enormous proportions that they are hard to work with.

- Usually these smells do not crop up right away, rather they accumulate over time as the program evolves (and especially when nobody makes an effort to eradicate them).

Bloaters (5)

- Long method
- Long class
- Primitive obsession
- Long parameter list
- Data Clumps



CamScanner

- **Long Method**
  - long methods are more difficult to understand
  - performance concerns with respect to lots of short methods are largely obsolete

- **Data Clumps**
  - attributes that clump together but are not part of the same class
    - Lack of cohesion – SRP
- **Primitive Obsession**
  - characterized by a reluctance to use classes instead of primitive data types

CamScanner

- Large Class
  - Large classes try to do too much, which reduces cohesion
    - violates SRP
- Long Parameter List
  - hard to understand, can become inconsistent
    - Symptom of behavioral form of god class

CamScanner

Object-orientation Abusers (4)

- Incomplete or incorrect application of object-oriented programming principles

- Switch statement
- Temporary field
- Refused bequest
- Alternative classes with different interfaces

599

- Switch Statements
  - Switch statements are often duplicated in code; they can typically be replaced by use of polymorphism (let OO do your selection for you!)
    - Typically a place where strategy or state pattern should have been used
- Temporary Field
  - An attribute of an object is only set in certain circumstances; but an object should need all of its attributes

600

- Refused Bequest
  - A subclass ignores most of the functionality provided by its superclass
    - violates LSP
- Alternative Classes with Different Interfaces
  - Symptom: Two or more methods do the same thing but have different signature for what they do

601

Software Design and Architecture

Topic#089

END!

602

Change Preventers (3)

Software Design and Architecture

Topic#090
Code Smells – Change Preventers

MCSQ + Subjective
by Sabahat Saad
Digital World VU

Change Preventers (3)

- These smells mean that if you need to change something in one place in your code, you have to make many changes in other places too.
- Program development becomes much more complicated and expensive as a result.

- Divergent change
- Shotgun surgery
- Parallel inheritance hierarchies

- Divergent Change
  - Deals with cohesion; symptom: one type of change requires changing one subset of methods; another type of change requires changing another subset
    - violates SRP

- Shotgun Surgery
  - a change requires lots of little changes in a lot of different classes

- Parallel Inheritance Hierarchies
  - Similar to Shotgun Surgery; each time I add a subclass to one hierarchy, I need to do it for all related hierarchies

CamScanner

Dispensables (5)

- A dispensable is something pointless and unneeded whose absence would make the code cleaner, more efficient and easier to understand.

- Comments
- Duplicated code
- Lazy class
- Data class
- Speculative generality

- Comments (!)
  - Comments are sometimes used to hide bad code
  - "...comments often are used as a deodorant" (!)

- Duplicated Code
  - bad because if you modify one instance of duplicated code but not the others, you (may) have introduced a bug!

- Lazy Class
  - Each class costs money to maintain and understand.
  - A class that no longer "pays its way"
    - e.g. may be a class that was downsized by refactoring, or represented planned functionality that did not pan out

CamScanner

- Data Class
  - These are classes that have fields, getting and setting methods for the fields, and nothing else; they are data holders, but objects should be about data AND behavior
    - Symptom of data form of god class

- Speculative Generality
  - "Oh I think we need the ability to do this kind of thing someday"

CamScanner

Couplers (5)

- All the smells in this group contribute to excessive coupling between classes or show what happens if coupling is replaced by excessive delegation.

- Feature envy
- Inappropriate intimacy
- Message chain
- Middle man
- Incomplete library class

- Message Chains
  - a client asks an object for another object and then asks that object for another object etc. Bad because client depends on the structure of the navigation
    - violates LoD

- **Feature Envy**
  - A method **requires lots of information from some other class** (move it closer!)
    - Symptom of behavioral form of god class

- **Inappropriate Intimacy**
  - Pairs of classes that know too much about each other's private details

BY Sabahat Saad (digital World VU)
for paid projects + lms handling
Contact us at 03261557791

- **Middle Man**
  - If a class is delegating more than half of its responsibilities to another class, do you really need it?

- **Incomplete Library Class**
  - A framework class doesn't do everything you need

CamScanner

Software Design and Architecture

Fakhar Lodhi

Topics Covered: 093 – 098

Refactoring – Part II

621

Refactoring

- How to fix a code smell

"A change to the system that leaves its behavior unchanged, but enhances some nonfunctional quality – simplicity, flexibility, understandability, performance"

(Kent Beck, Extreme Programming Explained, page 179)

623

CamScanner

The Catalog

- **name.** The name is important to building a vocabulary of refactorings.
- **summary** of the situation in which you need the refactoring
  - summary of what the refactoring does
- **motivation**
- **mechanics**
- **examples**

CamScanner

- Large Class
  - A class with too many instance variables or too much code
- Refactorings
  - Extract Class
  - Extract Subclass
  - Extract Interface
  - Replace Data Value with Object

CamScanner

The Catalog

- A few of the more common ones, include:
  - Extract Method
  - Replace Temp with Query
  - Move Method
  - Replace Conditional with Polymorphism
  - Introduce Null Object

CamScanner

Extract Method

- You have a code fragment that can be grouped together
  - Turn the fragment into a method whose name explains the purpose of the fragment

CamScanner

Replace Temp with Query

- You are using a temporary variable to hold the result of an expression
  - Extract the expression into a method
  - Replace all references to the temp with the expression
  - The new method can then be used in other methods

CamScanner

635

Replace Temp with Query

```
double basePrice = _quantity * _itemPrice;

if (basePrice > 1000)
  return basePrice * 0.95;
else
  return basePrice * 0.98;
```

637

```
double basePrice() {
    return _quantity * _itemPrice;
}
```

Replace Temp with Query

```
if (basePrice() > 1000)
  return basePrice() * 0.95;
else
  return basePrice() * 0.98;
...
```

639

Software Design and Architectu

Topic#095

END!

Software Design and Architecture

Move Method

CamScanner

Move Method

- A method is using more features (attributes and operations) of another class than the class on which it is defined
  - Create a new method with a similar body in the class it uses most.
  - Either turn the old method into a simple delegation, or remove it altogether

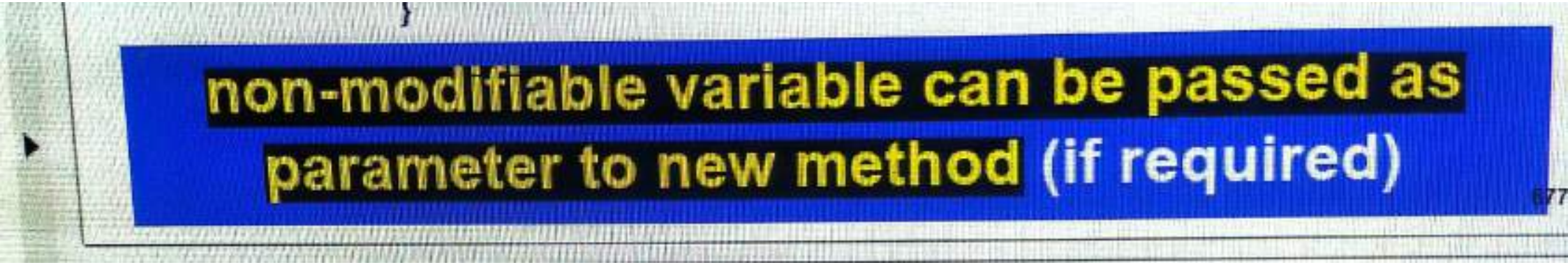
CamScanner

Replace Conditional with Polymorphism

- You have a conditional that chooses different behavior depending on the type of an object
- Move each "leg" of the conditional to an overriding method in a subclass.
- Make the original method abstract

MCQS
by Sabahat Saad

CamScanner


non-modifiable variable can be passed as
parameter to new method (if required)

CamScanner

modified variables require more care; since there is only one, we can make it the return value of the new method.

CamScanner

Why Developers are Reluctant to Refactor

- Lack of understanding
- Short-term focus
- Not paid for overhead tasks like refactoring
- Fear of breaking current program

CamScanner

Software Design and Architecture

Fakhar Lodhi

Topics Covered: 110 – 118

MVC Architecture

CamScanner

- A new application has to be developed

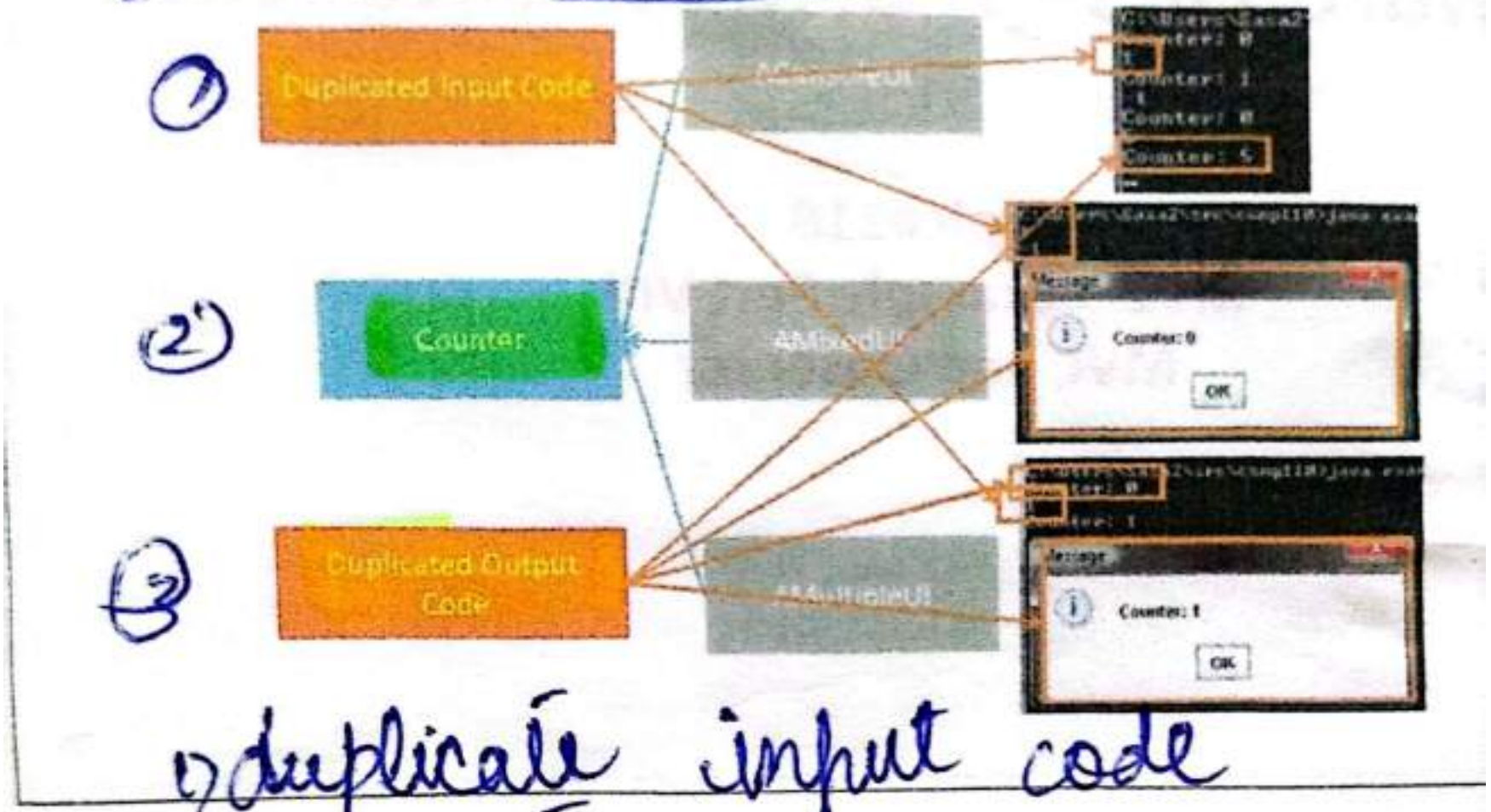
- Now we have three application
  1. Console-based UI
  2. Mixed UI
  3. Multiple Mixed UI

Models and Interactors

- Model:
  - common functionality
  - counter semantics: storage and manipulation of an integer counter
- Interactor
  - different for each interface
  - counter user-Interface: user-interface to display and change the counter

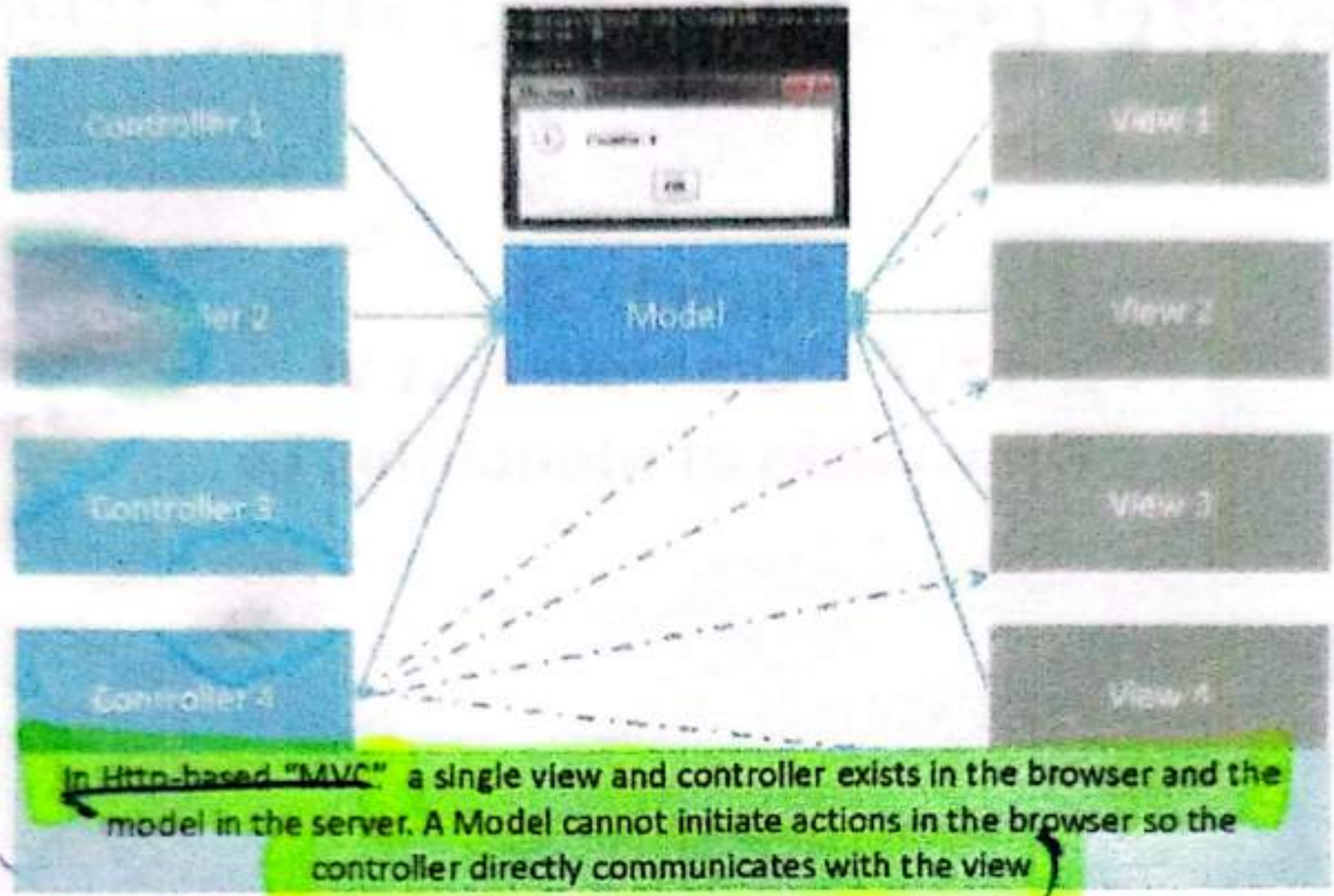
CamScanner

Drawbacks of Monolithic UI ?



1) duplicate input code
2) Counter
3) Duplicate output code.

yncing Controllers & View



In Http-based "MVC" a single view and controller exists in the browser and the model in the server. A Model cannot initiate actions in the browser so the controller directly communicates with the view

Changing to Console View



The architecture is a bridge between those (often abstract) business goals and the final (concrete) resulting system.

MCQs

Jabbar Saad
Dey digital World JU

Path from abstract goals to concrete systems

For Paid : 0326-1557791

803

Software Architecture? Defined

Subjective

- The software architecture of a system is the set of structures needed to reason about the system

MCQs

- Comprises of
  - Software elements
  - Relations among them
  - Properties of both

806

Software Architecture Defined

Some people define the architecture as the system's "early" or "major" design decisions MCQs.

Software Architecture as a set of Structures

- Structure - a set of elements held together by a relation
- Software systems are composed of many structures
- No single structure holds claim to being *the* architecture

---

Q How many Categories of Structures in Architectural Design?

- Three categories MCQs + Subjective

BY → Sabahat Saad
YouTube → Digital World VU
WhatsApp → 0326-1557791
Contact for Paid Project & LMS Handle

CamScanner

Categories of Structures in Architectural Design

- Static Structures
  - Implementation units or modules
- Dynamic Structures
  - *component-and-connector* (C&C) structures – components are runtime entities
- Deployment Structures
  - aka allocation structures

Categories of Structures

- Play an important role in the design, documentation, and analysis of architectures

(1) Implementation units or modules

- Static structures
  - focus on the way the system's functionality is divided up and assigned to implementation teams.

⇒ Assigned specific computational responsibilities

- Basis of work assignments for programming teams
  - (Team A works on the database, Team B works on the business rules, Team C works on the user interface, etc.)

CamScanner

Implementation units or modules

- In large projects, these elements (modules) are subdivided for assignment to subteams.
- For example, the database for a large enterprise resource planning (ERP) implementation might be so complex that its implementation is split into many parts.
- aka Module Decomposition Structure
  - captures that decomposition

CamScanner

Implementation units or modules

- Class Diagrams MCQs
  - Kind of module structure emerges as an output of object-oriented analysis and design
- Layered Structure MCQs
  - Modules aggregated (organized) into layers

CamScanner

Dynamic Structures

2 Component-and-Connector (C&C) Structures

- Runtime structures
  - Component
- Focus on the way the elements interact with each other at runtime to carry out the system's functions.

CamScanner

Allocation Structures

- Mapping from software structures to the system's organizational, developmental, installation, and execution environments

MCQs

Components are deployed onto hardware in order to execute

CamScanner

Software Design and Architecture

Topic#123
Quality Attributes

CamScanner

- A structure is architectural if it supports reasoning about the system and the system's properties
  - an attribute of the system that is important to some stakeholder MCQs

The set of architectural structures is not fixed or limited

What is architectural is what is useful in your context for your system

826

- These include * Subjective
  - functionality achieved by the system
  - the availability of the system in the face of faults
  - the difficulty of making specific changes to the system
  - the responsiveness of the system to user requests
  - and many others.

Quality Attributes

827

CamScanner

Software Architecture

- an architecture is an *abstraction* of a system that selects certain details and suppresses others.
- The architecture specifically omits certain information about elements that is not useful for reasoning about the system
  - information that has no ramifications outside of a single element

CamScanner

Software Architecture

- Architecture is concerned with the public interface MCOs.
  - private details of elements—details having to do solely with internal implementation—are not architectural.

CamScanner

MCQs.

Every Software System Has a Software Architecture

- Every system can be shown to comprise elements and relations among them to support some type of reasoning.
  - In the most trivial case, a system is itself a single element—an uninteresting and probably non-useful architecture, but an architecture nevertheless.

CamScanner

Software Design and Architecture

Topic#126
Difference between Software, System, and Enterprise Architectures

839

System Architecture

- Concerned with a total system, including hardware, software, and humans
- mapping of functionality onto hardware and software components
  - a mapping of the software architecture onto the hardware architecture, and a concern for the human interaction with these components

842

CamScanner

System Architecture

- A description of the software architecture, as it is mapped to hardware and networking components, allows reasoning about qualities such as performance and reliability
- A description of the system architecture will allow reasoning about additional qualities such as power consumption, weight, and physical footprint

CamScanner

Enterprise Architecture? Subjective

A description of the structure and behavior of an organization's processes, information flow, personnel, and organizational subunits, aligned with the organization's core goals and strategic direction

May or may not include information systems

MCQs

CamScanner

• View is a representation of a structure.

Structures and Views

- A view is a representation of a coherent set of architectural elements, as written by and read by system stakeholders
- consists of a representation of a set of elements and the relations among them.

CamScanner

"4+1 view model"

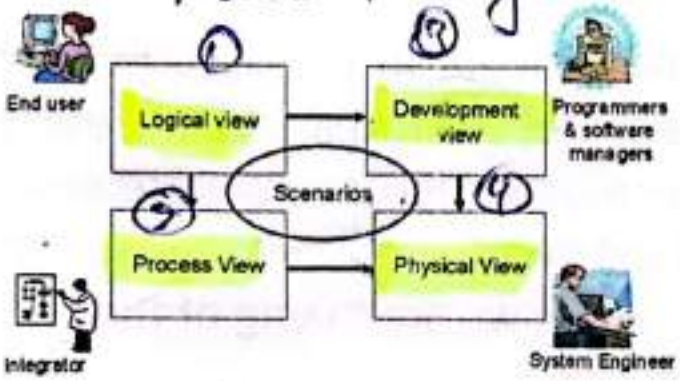
- Philippe Kruchten
  - leader of RUP development team in Rational corp. (now owned by IBM)
  - Valuable experiences in industry (Telecom, Air traffic control system) which he used them for confirmation of his model

857

4+1 View Model of Architecture

Most Imp

MCQs + Subjective



860

(3) Development View

Subjective

(Subsystem decomposition)

Basis of a line of product

Viewer: Programmers and Software Managers

considers:

- software module organization
- (Hierarchy of layers, software management, reuse, constraints of tools)

863

CamScanner

(i) Logical View

Subjective

(Object-oriented Decomposition)

Viewer: End-user

considers:

- Functional requirements
  - What the system should provide in terms of services to its users.

(ii) Physical View

Subjective.

(Mapping the software to the Hardware)

Viewer: System Engineers

Considers:

- Non-functional req. regarding underlying hardware (Topology, Communication)

CamScanner

(2)

Process View

Subjective

(The process decomposition)

viewer: Integrators

considers:

- Non - functional requirements
  - (concurrency, performance, scalability)

CamScanner

Sc

Scenarios

(Putting it all together)

Viewer: All users of other views and Evaluators.

Considers: System consistency, validity

all modules/views
(1-4)

CamScanner

Correspondence between views

- Views are interconnected.
- Start with Logical view (Req. Doc) and Move to Development or Process view and then finally go to Physical view.

Q: The Three Categories of Structures in Architectural Design ?

Subjective

1. Static Structures
   - Module Structures
2. Dynamic Structures
   - *component-and-connector* (C&C) structures – components are runtime entities
3. Deployment Structures
   - aka allocation structures

CamScanner

Module Structures

- Embody decisions as to how the system is to be structured as a set of code or data units that have to be constructed or procured.
- In any module structure, the elements are modules of some kind (perhaps classes, or layers, or merely divisions of functionality, all of which are units of implementation).

CamScanner

Subjective.

Module Structures

MCQs

- Modules represent a static way of considering the system.
- Modules are assigned areas of functional responsibility ,
  - there is less emphasis in these structures on how the resulting software manifests itself at runtime.

BY: Sabahat Saad Digital World VU.

CamScanner

Module Structures

- Module structures allow us to answer questions such as these:
  - What is the primary functional responsibility assigned to each module?
  - What other software elements is a module allowed to use?
  - What other software does it actually use and depend on?
  - What modules are related to other modules by generalization or specialization
    - (i.e., inheritance) relationships?

Module Structures

Subjective

- Module structures convey this information directly, but they can also be used by extension to ask questions about the impact on the system when the responsibilities assigned to each module change.
  - looking at its module views is an excellent way to reason about a system's modifiability

CamScanner

Subjective.

(2) Component-and-connector Structures

- Embody decisions as to how the system is to be structured as a set of elements that have runtime behavior (components) and interactions (connectors)
- The elements are runtime components
  - which are the principal units of computation
    - services, peers, clients, servers, filters, etc
  - connectors - communication vehicles among components
    - call-return, process synchronization operators, pipes, etc

CamScanner

(3)

Allocation Structures

- Embody decisions as to how the system will relate to non software structures in its environment
  - CPUs, file systems, networks, development teams, etc.
- These structures show the relationship between the software elements and elements in one or more external environments in which the software is created and executed.

MCOs + Subsystem

CamScanner

Uses of allocation structure

Useful for

1. Performance, availability, security analysis
2. Configuration control, integration, test activities
3. Project management, best use of expertise and available resources, management of commonality

CamScanner

Software Design and Architectur

Topic#135
Architectural Patterns – Overview

Some Examples

- Module Patterns ①
  - Layered Pattern
- Component-and-connector type pattern ②
  - Shared Data (or repository) Pattern
  - Client-Server Pattern
- Allocation Patterns ③
  - Multi-tier pattern
  - Competence center and platform center

Client-Server Pattern

②

- Component-and-connector type pattern
- The components are the clients and the servers, and the connectors are protocols and messages they share among each other to carry out the system's work.

Layered Pattern

Subjective

- *Module type pattern*
- When the *uses* relation among software elements is strictly unidirectional, a system of layers emerges. MCQs
- A layer is a coherent set of related functionality.
- In a *strictly* layered structure, a layer can only use the services of the layer immediately below it.
- Many variations of this pattern, lessening the structural restriction, occur in practice. MCQs
- Layers are often designed as abstractions (virtual machines) that hide implementation specifics below from the layers above, engendering portability

(1)

Shared Data (or repository) Pattern

Shared data Pattern is also called repository Pattern

(2)

- Component-and-connector type pattern
- comprises components and connectors that create, store, and access persistent data.
- The repository usually takes the form of a database.
- The connectors are protocols for managing the data, such as SQL.

MCQs

CamScanner

Multi-tier Pattern

- Allocation pattern
- Describes how to distribute and allocate the components of a system in distinct subsets of hardware and software, connected by some communication medium.
- This pattern specializes the generic deployment (software-to-hardware allocation) structure.

Competence center and platform

- Allocation pattern
- specialize a software system's work assignment structure.
- In *competence center*, work is allocated to sites depending on the technical or domain expertise located at a site.
  - For example, user-interface design is done at a site where usability engineering experts are located.
- In *platform*, one site is tasked with developing reusable core assets o a software product line, and other sites develop applications that use the core assets.

CamScanner

MCQs

- There are rules of thumb that should be followed when designing most architectures
- Failure to apply any of these does not automatically mean that the architecture will be fatally flawed, but it should at least serve as a warning sign that should be investigated.

CamScanner

Structural Recommendations - 1

- The architecture should feature well-defined modules whose functional responsibilities are assigned on the principles of information hiding and separation of concerns.
  - The information-hiding modules should encapsulate things likely to change, thus insulating the software from the effects of those changes.
  - Each module should have a well-defined interface that encapsulates or "hides" the changeable aspects from other software that uses its facilities.
  - These interfaces should allow their respective development teams to work largely independently of each other.

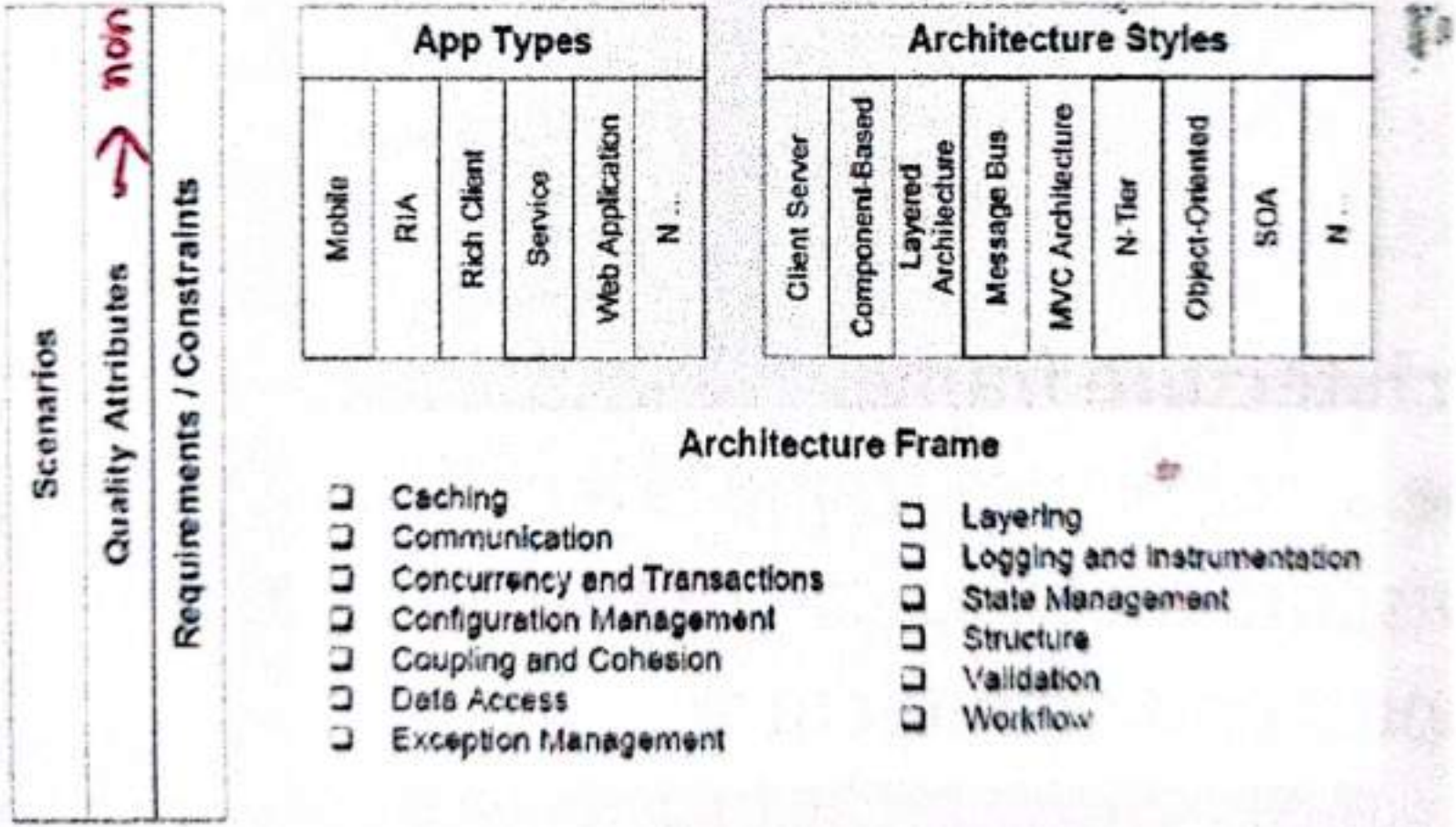
Structural Recommendations - 4

MCOs

- Modules that produce data should be separate from modules that consume data.
  - This tends to increase modifiability because changes are frequently confined to either the production or the consumption side of data.
- If new data is added, both sides will have to change, but the separation allows for a staged (incremental) upgrade.

CamScanner

The Architecture Meta-Frame



Software Design and Architecture

Topic#141
The Architecture Meta-Frame
Quality Attributes: Introduction

CamScanner

- A quality attribute (QA) is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders.

eg Performance (non-fun requir)

- a quality attribute is a measure of the "goodness" of a product along some dimension of interest to a stakeholder

BY: Sabahat Saeed (digital image VU)

CamScanner

NCOI

- architecture provides the mapping of a system's functionality onto software structures that determines the architecture's support for qualities

- how various qualities are supported by architectural design decisions

CamScanner

Software Design and Architecture

Topic#144
The Architecture Meta-Frame
Quality Attributes – Description

CamScanner

Category	Description
Availability (1)	Availability defines the proportion of time that the system is functional and working. It can be measured as a percentage of the total system downtime over a predefined period. Availability will be affected by system errors, infrastructure problems, malicious attacks, and system load.
Conceptual integrity (2)	Conceptual integrity defines the consistency and coherence of the overall design. This includes the way that components or modules are designed, as well as factors such as coding style and variable naming.

Obj + Subje
Most Imp

Total
(13 points)

CamScanner

Category	Description
Flexibility (3)	Flexibility is the ability of a system to adapt to varying environments and situations, and to cope with changes to business policies and rules. A flexible system is one that is easy to reconfigure or adapt in response to different user and system requirements.
Interoperability (4)	Interoperability is the ability of diverse components of a system or different systems to operate successfully by exchanging information, often by using services. An interoperable system makes it easier to exchange and reuse information internally as well as externally.
Maintainability (5)	Maintainability is the ability of a system to undergo changes to its components, services, features, and interfaces as may be required when adding or changing the functionality, fixing errors, and meeting new business requirements.

Imp obj + subje

CamScanner

Manageability (6)	Manageability defines how easy it is to manage the application, usually through sufficient and useful instrumentation exposed for use in monitoring systems and for debugging and performance tuning.
Performance (7)	Performance is an indication of the responsiveness of a system to execute any action within a given interval of time. It can be measured in terms of latency or throughput. *Latency* is the time taken to respond to any event. *Throughput* is the number of events that take place within given amount of time.
Reliability (8)	Reliability is the ability of a system to remain operational over time. Reliability is measured as the probability that a system will not fail to perform its intended functions over a specified interval of time.

MCQ + Subj

CamScanner

Reusability	Reusability defines the capability for components and subsystems to be suitable for use in other applications and in other scenarios. Reusability minimizes the duplication of components and also the implementation time.
Scalability (9)	Scalability is the ability of a system to function well when there are changes to the load or demand. Typically, the system will be able to be extended by scaling up the performance of the server, or by scaling out to multiple servers as demand and load increase.
Security (10)	Security defines the ways that a system is protected from disclosure or loss of information, and the possibility of a successful malicious attack. A secure system aims to protect assets and prevent unauthorized modification of information.

MD + Sudy

CamScanner

Supportability (11)	Supportability defines how easy it is for operators, developers, and users to understand and use the application, and how easy it is to resolve errors when the system fails to work correctly.
Testability (12)	Testability is a measure of how easy it is to create test criteria for the system and its components, and to execute these tests in order to determine if the criteria are met. Good testability makes it more likely that faults in a system can be isolated in a timely and effective manner.
Usability (13)	Usability defines how well the application meets the requirements of the user and consumer by being intuitive, easy to localize and globalize, able to provide good access for disabled users, and able to provide a good overall user experience.

Suley + McCl.

CamScanner

Software Design and Architecture

Topic#145
The Architecture Meta-Frame
Requirements and Constraints

Requirements
drive architecture

(use cases, user stories, features, etc)

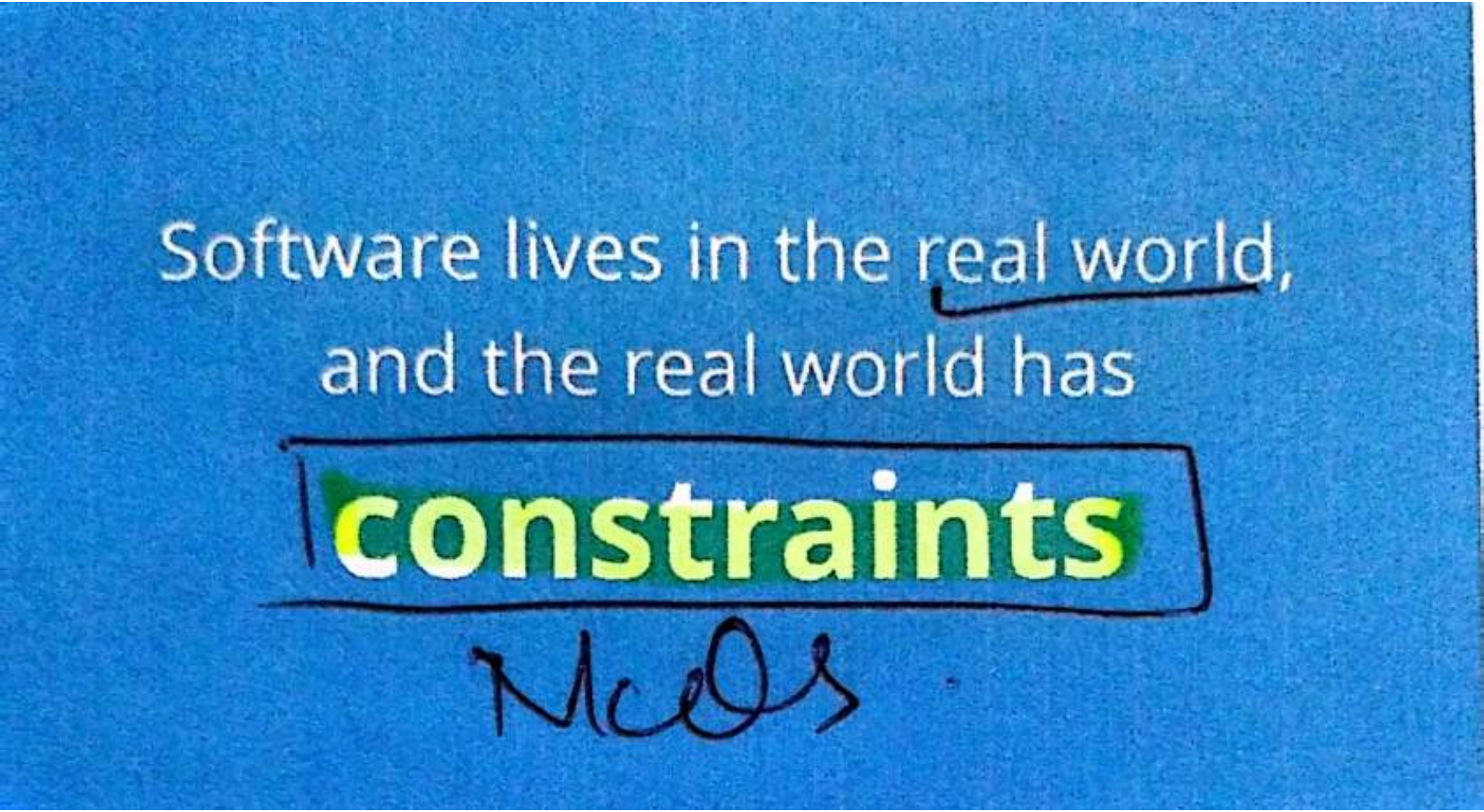
Q: Define Constraints

- Constraints MCOs
  - a design decision with zero degrees of freedom
  - external factors (such as not being able to train the staff in a new language, or having a business agreement with a software supplier, or pushing business goals of service interoperability) have led those in power to dictate these design outcomes.

MCOs

Typical constraints include time and budget, technology, people and skills, politics, etc

CamScanner


Software lives in the real world,
and the real world has
constraints
Needs

CamScanner

Application Types

- Your choice of an appropriate application type is governed by your specific requirements and infrastructure limitations.
- Use scenarios to help you choose an application type.

MCQs

1004

Application Types

MCQs · Quiz

- For example, if you want to support rich media and graphics delivered over the Internet, a rich Internet application (RIA) is probably the best choice.
- However, if you want to support data entry with forms in an occasionally connected scenario, a rich client is probably the best choice.

CamScanner

architecting an application.

1003

Application Type

Imp for Exams.

1. Mobile applications designed for mobile devices.
2. Rich client applications designed to run primarily on a client PC.
3. Rich Internet applications designed to be deployed from the Internet, which support rich user interface (UI) and media scenarios.

1006

Application Type

4. Service applications designed to support communication between loosely coupled components.
5. Web applications designed to run primarily on the server in fully connected scenarios.

CamScanner

Deployment Strategy

use of dep strat

- This helps you to choose an appropriate deployment topology, and to resolve conflicts between the application and infrastructure architecture early in the process.

short + MCQ

1014

Application type	Description
Mobile Application	• Can be developed as a Web application or a rich client application. • Can support occasionally connected scenarios. • Runs on devices with limited hardware resources.
Rich Client Application	• Usually developed as a stand-alone application. • Can support disconnected or occasionally connected scenarios. • Uses the processing and storage resources of the local machine.
Rich Internet Application	• Can support multiple platforms and browsers. • Can be deployed over the Internet. • Designed for rich media and graphical content. • Runs in the browser sandbox for maximum security. • Can use the processing and storage resources of the local machine.

Most Imp.

1017

CamScanner

Service Application	• Designed to support loose coupling between distributed components. • Service operations are called using XML-based messages. • Can be accessed from the local machine or remotely, depending on the transport protocol.
Web Application	• Can support multiple platforms and browsers. • Supports only connected scenarios. • Uses the processing and storage resources of the server.

Service applicant

By Digital World VU
For Paid Projects + LMS Handling
Contact US at : 0326-1557791

1018

Architectural Style

- aka architectural pattern
- The choice of architectural styles represents a set of principles that a design will follow
  - an organizing set of ideas that can be used to keep the design cohesive and focused on the key objectives and scenarios.

1021

CamScanner

Architectural Style

- Examples of architectural styles are:
  - client/server, component-based, layered architecture, message-bus, Separated Presentation, 3-tier/N-tier, object-oriented, and service-oriented architecture (SOA).

CamScanner

Architectural Styles

- choice of architectural styles depends upon
  1. application type
  2. the requirements and constraints
  3. the scenarios you want to support, and
  4. the styles with which one is most familiar and comfortable.

CamScanner

Architecture Frame

- Each hotspot represents key engineering decisions
- Each represents an opportunity to improve your design and build a technically more effective architecture.

Cross-Cutting Concerns

- Cross-cutting concerns represent key areas of your design that are not related to a specific layer in your application.
- For example, you might want to cache data in the presentation layer, the business layer, and the data access layer.

CamScanner

Key Cross-Cutting Concerns and authorization

- allow one to identify the users of an application with confidence, and to determine the resources and operations to which they should have access.

Key Cross-Cutting Concerns

- **Authentication.**
  - Determine how to authenticate your users and pass authenticated identities across the layers.

Key Cross-Cutting Concerns - Caching

- Caching improves performance, reduces server round trips, and can be used to maintain the state of your application.
- Identify what should be cached, and where to cache, to improve application's performance and responsiveness.

Key Cross-Cutting Concerns - Communication

- Communication strategies determine how will communication be achieved between layers and tiers, including protocol, security, and communication-style decisions.
- Choose appropriate protocols, reduce calls across the network, and protect sensitive data passing over the network.

Key Cross-Cutting Concerns

- **Authorization**
  - Ensure proper authorization with appropriate granularity within each layer, and across trust boundaries.

1038

Key Cross-Cutting Concerns – Exception Management

- Exception-management strategies describe techniques for handling errors, logging errors for auditing purposes, and notifying users of error conditions.
- **Exception management**. Catch exceptions at the boundaries. Do not reveal sensitive information to end users.

1041

3

- Data access strategies describe techniques for abstracting and accessing data in your data store.
- This includes
  - data entity design,
  - error management, and
  - managing database connections.

CamScanner

Q: Write down

Key Architecture Principles? 4 Principles

1. **Build to change over build to last.**
   - Wherever possible, design your application so that it can change over time to address new requirements and challenges.
2. **Model to analyze and reduce risk.**
   - Use threat models to understand risks and vulnerabilities. MCQs

1078

Key

3. **Mode**
   collal
   - Effi
     des
   - Use
     con
     rap

1078

Key Architecture Principles

- **Models and views are a communication and collaboration tool.**
  - Efficient communication of design principles and design changes is critical to good architecture.
  - Use models and other visualizations to communicate your design efficiently and to enable rapid communication of changes to the design.

1079

Key Architecture Principles

- **Identify key engineering decisions.**
  - Use the architecture frame in this guide to understand the key engineering decisions and tl areas where mistakes are most often made.
  - Invest in getting these key decisions right the fir time so that the design is more flexible and less likely to be broken by changes.

CamScanner

Baseline Architecture

- A *baseline architecture* describes the existing system
  - it is how your system looks today.
- If this is a new architecture, your initial baseline is the first high-level architectural design from which candidate architectures will be built.

diff b/w baseline & candidate architecture

Candidate Architecture

- A candidate architecture includes the application type, the deployment architecture, the architectural style, technology choices, quality attributes, and cross-cutting concerns.

Contact us for paid project + LMS Hack
0326-1557791

Notes By: Miss Sabahat Saad
Youtube: Digital World VU.

CamScanner

Software Design and Architecture

Topic#162
Architectural Spikes

Architectural Spikes

- An *architectural spike* is an end-to-end test of a small segment of the application.
- The purpose of an architectural spike is to reduce risk and to test potential paths.

Architectural Spikes

- If the candidate architecture is an improvement, it can become the new baseline from which new candidate architectures can be created and tested.

Architectural Spikes

- This iterative and incremental approach allows you to get the big risks out of the way first, iteratively render your architecture, and use architectural tests to prove that each new baseline is an improvement over the last.

Architectural Spikes

- As you evolve your architecture, you may use spikes to explore different scenarios without impacting the existing design.
- An architectural spike will result in a candidate architecture that can be tested against a baseline.

CamScanner

Reference application architecture

- Represents a canonical view of a typical application architecture, using a layered style to separate functional areas into separate layers.
- The reference application architecture demonstrates how a typical application might interact with its users, external systems, data sources, and services.

CamScanner

Software Architecture Patterns
aka architecture styles

- A **software architecture pattern** is a solution to a recurring problem that is well understood, in a particular context.
- Each pattern consists of a context, a problem, and a solution.

CamScanner

Software Architecture Patterns

MCQS.

- Each pattern has its own characteristics, strengths, and weaknesses.

diff b/w software archi pattern & design pattern

- Software architecture patterns are similar to design patterns, except that they are broader in scope and are applied at the architecture level.

(2)

CamScanner

diff (Subjective)

(3) Architecture patterns tend to be more coarse-grained and focus on architectural problems while design patterns are more fine-grained and solve problems that occur during implementation.

(Subjective)

(4)
- A software architecture pattern provides a high-level structure and behavior for software systems.
- It is a grouping of design decisions that have been repeated and used successfully for a given context.

(5)
- Software architecture patterns provide the structure and main components of the software system being built.

(6) architectural pattern
- They introduce design constraints, which reduce complexity and help to prevent incorrect decisions.

CamScanner

- When a software architecture pattern is followed consistently during design, we can anticipate the properties that the software system will exhibit.
- This allows us to consider whether a design will satisfy the requirements and quality attributes of the system.

- Software architecture patterns can be applied to the entire software system or to one of the subsystems.
- Consequently, more than one software architecture pattern can be used in a single software system. These patterns can be combined to solve problems.

Software Design and Architecture

Topic#167
Types of Architecture Patterns

The Three Categories of Structures in Architectural Design

1. Static Structures
   - Module Structures
2. Dynamic Structures
   - *component-and-connector* (C&C) structures – components are runtime entities
3. Deployment Structures
   - aka allocation structures

The Three Categories of Structures in Architectural Design

- Static Structures
  - Module Structures
    - assigned areas of functional responsibility
    - how the system is to be structured as a set of code or data units that have to be constructed or procured

Miss Sabahat Saad
for paid Assessments
0326-1557791

The Three Categories of Structures in Architectural Design

- *component-and-connector* (C&C) structures – components are runtime entities
  - how the system is to be structured as a set of elements that have runtime behavior (components) and interactions (connectors)
  - components - the principal units of computation
  - connectors - communication vehicles among components

The Three Categories of Structures in Architectural Design

- Deployment Structures
  - how the system will relate to non-software structures in its environment
  - show the relationship between the software elements and elements in one or more external environments in which the software is created and executed

CamScanner

1130

1131

- Patterns can be categorized by the dominant type of elements that they show
  - module patterns show modules,
  - component-and-connector (C&C) patterns show components and connectors
  - allocation patterns show a combination of software elements (modules, components, connectors) and non-software elements.

1133

1134

Software Design and Architecture

Topic#168
Module Patterns
Layered Pattern – I: Introduction

- **Solution:**
  - the layered pattern divides the software into units called layers.
  - Each layer is a grouping of modules that offers a cohesive set of services.

CamScanner

- one of the most common techniques.
- In a **layered architecture** the software application is divided into various horizontal layers, with each layer located on top of a lower layer.

CamScanner

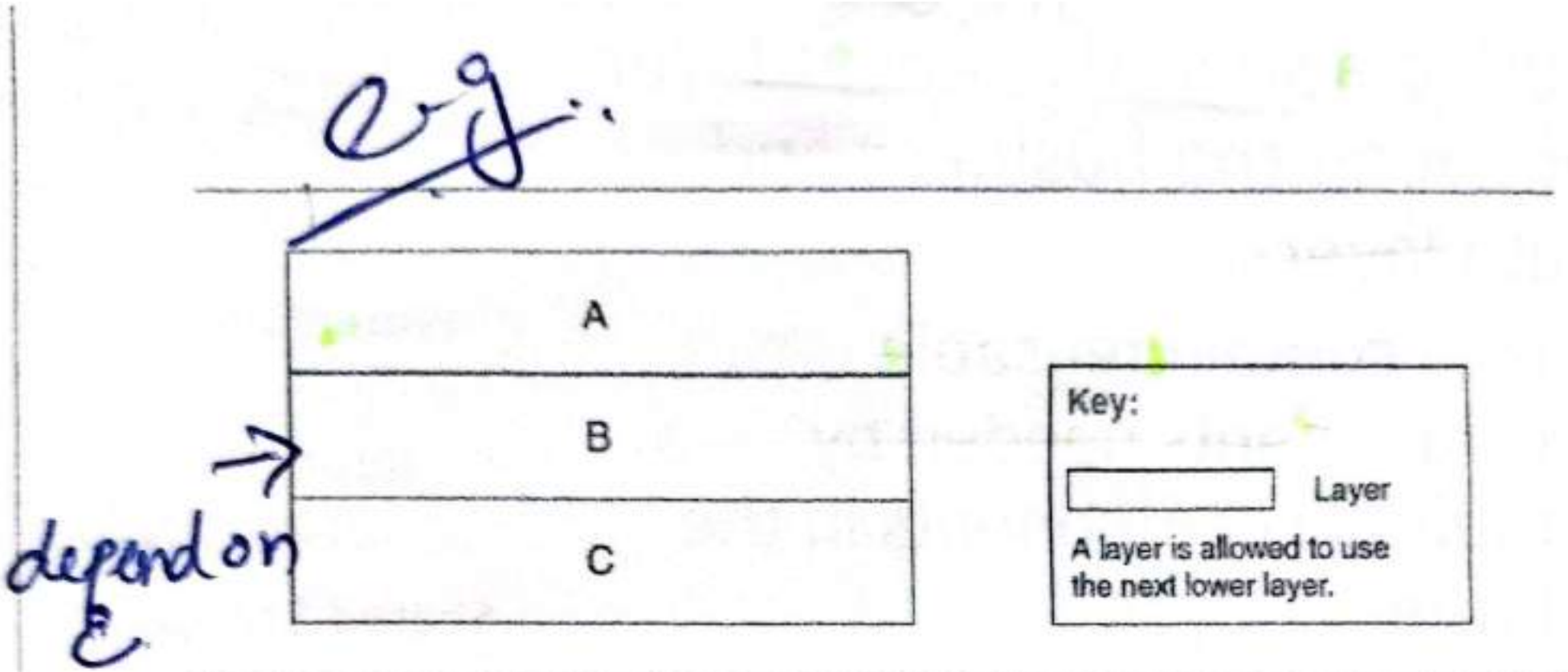


FIGURE 13.1 Stack-of-boxes notation for layered designs

CamScanner

- Each layer is dependent on one or more layers below it (depending on whether the layers are open or closed), but is independent of the layers above it.

CamScanner

Software Design and Architecture

Topic#169
Module Patterns
Layered Pattern – II: Open vs Closed Layers

Q: Diff b/w closed & Open layer.

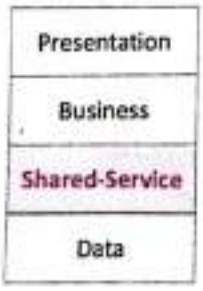
- **Open versus closed layers**
  - With a closed layer, requests that are flowing down the stack from the layer above must go through it and cannot bypass it.

- Closed layers provide layers of isolation, which makes code easier to change, write, and understand.
- This makes the layers independent of each other, such that changes made to one layer of the application will not affect components in the other layers.

- If the layers are open, this increases complexity.
- Maintainability is lowered because multiple layers can now call into another layer, increasing the number of dependencies and making changes more difficult.

MCQs

- added a shared services layer between the business and data layers.
  - may contain reusable components needed by multiple components in the business layer.
  - placed it below the business layer so that only the business layer has access to it.



MCQs

- all requests from the business layer to the data layer must go through the shared services layer even though nothing is needed from that layer.

If closed →



CamScanner

Q: Write down closed layer advantages

- If the shared services layer was made open (aka layer bridging), requests to the data layer can be made directly from the business layer.

open layer →



- there are advantages to closed layers and achieving layers of isolation.
1. However, it might be appropriate to open a layer.
2. It is not necessary to make all of the layers open or closed.
3. You may selectively choose which layers, if any, are open.



Software Design and Architecture

Topic#170
Module Patterns
Layered Pattern – III: Tiers vs Layers

- Layers – Logical separation
- Tiers – Physical separation

MCQs

- When a software architecture is organized into more than one layer, it is known as a multi-layer architecture. MCQs
- Different layers do not necessarily have to be located on different physical machines.
- It is possible to have multiple layers on the same machine.

- Tiers concern themselves with the physical location of the functionality and components. MCQs

CamScanner

- When a software architecture is partitioned into multiple tiers, it is known as a **multi-tier architecture**.

MCQs

1182

Advantages of layered architecture?

MCQs + Subj.

1. This pattern reduces complexity by achieving a Separation of Concerns (SoC).
   - Each layer is independent and you can understand it on its own without the other layers.
2. Complexity can be abstracted away in a layered application, allowing us to deal with more complex problems.

1185

(3) • Dependencies between layers can be minimized in a layered architecture, which further reduces complexity.
  • For example, the presentation layer does not need to depend directly on the data layer and the business layer does not depend on the presentation layer.

(4) • Minimizing dependencies also allows you to substitute implementations for a particular layer without affecting the other layers.

BY: Miss Sabahat Saad
For Paid → 0326-1557791

(5) they can make development easier
  • The pattern is pervasive and well known to many developers, which makes using it easy for the development team.
  • Due to the way that the architecture separates the application logic, it matches up well with how many organizations hire their resources and allocate tasks during a project.
  • Each layer requires a particular skill set and suitable resources can be assigned to work on each layer.

(6) • This architecture pattern increases the testability quality attribute of software applications.
  • Partitioning the application into layers and using interfaces for the interaction between layers allows us to isolate a layer for testing and either mock or stub the other layers.

MCQs

  • For example, you can perform unit testing on classes in your business layer without the presentation and data layers.
  • The business layer is not dependent on the presentation layer and the data layer can be mocked or stubbed.

CamScanner

(7)
- Applications using a layered architecture may have higher levels of reusability if more than one application can reuse the same layer.
  - For example, if multiple applications target the same business and/or data layers, those layers are reusable.

(8)
- When an application using a layered architecture is deployed to different tiers, there are additional benefits:

CamScanner

Q: Disadvantage of layer architecture

1. Although the layers can be designed to be independent, a requirement change may require changes in multiple layers.
2. This type of coupling lowers the overall agility of the software application.
   - For example, adding a new field will require changes to multiple layers: the presentation layer so that it can be displayed, the business layer so that it can be validated/saved/processed, and the data layer because it will need to be added to the database.

175

1176

3. there can be inefficiencies in having a request go through multiple layers.

CamScanner

- This can complicate deployment because, even for a change such as this, an application may require multiple parts (or even the entire application) to be deployed.
- more code will be necessary for layered applications.
  - This is to provide the interfaces and other logic that are necessary for the communication between the multiple layers.

- Development teams have to be diligent about placing code in the correct layer so as not to leak logic to a layer that belongs in another layer.
  - Examples of this include placing business logic in the presentation layer or putting data-access logic in the business layer.

CamScanner

(7)

- moving from one layer to another sometimes requires data representations to be transformed.

1180

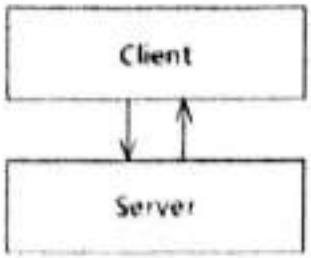
MCQs

- There is a greater monetary cost associated with having a multi-tier architecture.
  - The more machines are used for the application, the greater the overall cost.
  - Unless the hosting of the software application is handled by a cloud provider or has otherwise been outsourced, an internal team will be needed to manage the physical hardware of a multi-tier application.

1183

Also Imp 4 Subjective MCQs

- client-server architecture, also known as a two-tier architecture
- distributed application
- clients and servers communicate with each other directly.
- A client requests some resource or calls some service provided by a server and the server responds to the requests of clients.
- There can be multiple clients connected to a single server

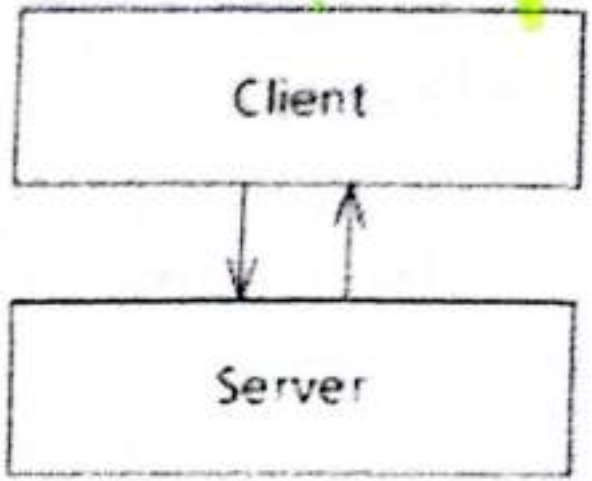


like client & server model

1186

CamScanner

- The **Client** part of the application contains the user interface code and the **Server** contains the database.
- The majority of application logic in a client-server architecture is located on the server, but some of it could also be located in the client.
- The application logic located on the server might exist in software components, in the database, or both.

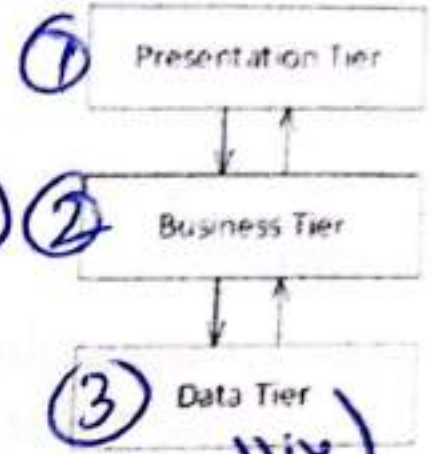


MCQs

CamScanner

n-Tier Structure

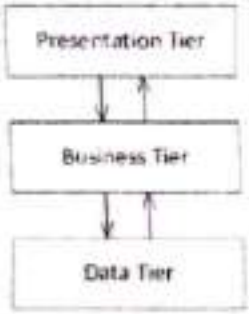
- also known as a **multitier architecture**
- there are multiple tiers in the architecture.
- One of the most widely-used variations of this type of layered architecture is the three-tier architecture.
- The three-tier architecture separates logic into presentation, business, and data layers:



By: Sabahat Saad
for paid (Project + LNShandling)
contact us at: 0326-1557791
CamScanner

- **Presentation tier**

- The **presentation tier** provides functionality for the application's UI.
- It should provide an appealing visual design as it is the part of the application that users interact with and see.
- Data is presented to the user and input is received from users in this tier.

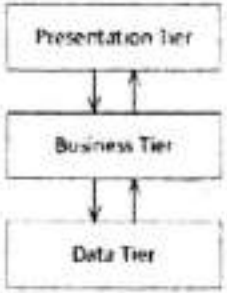


- **Presentation tier**

- Aspects of the usability quality attribute should be the concern of the presentation tier.
- Software architects should strive to design *thin clients* that minimize the amount of logic that exists in the presentation tier.

CamScanner

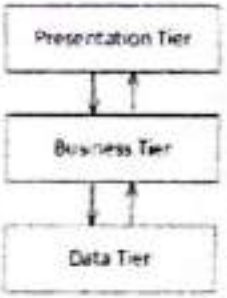
- **Presentation tier**
- The logic in the presentation tier should focus on user interface concerns.
- A presentation tier devoid of business logic will be easier to test.



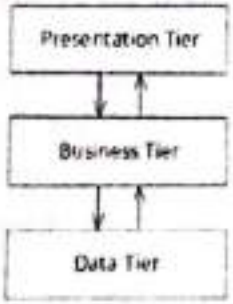
1198

(2)

- **Business tier**
- The **business tier**, which is sometimes referred to as the application tier, provides the implementation for the business logic of the application, including such things as business rules, validations, and calculation logic.
- Business entities for the application's domain are placed in this tier.



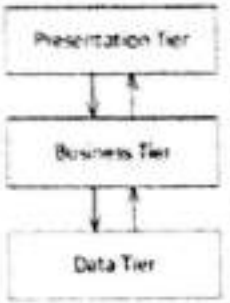
- **Business tier**
- It provides the presentation tier with services, commands, and data that it can use, and it interacts with the data tier to retrieve and manipulate data.



1201

(3) **Data tier** Subjective + Objective

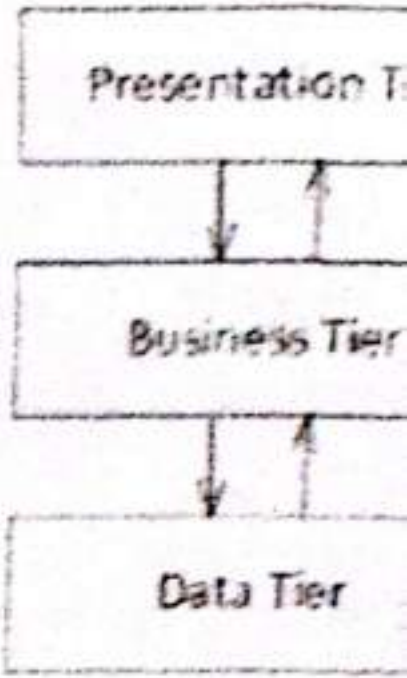
- The **data tier** provides functionality to access and manage data.
- The data tier contains a data store for persistent storage, such as an RDBMS.
- It provides services and data for the business tier.



1202

CamScanner

- **Business tier**
- The business tier coordinates the application and executes logic.
- It can perform detailed processes and makes logical decisions.
- The business tier is the center of the application and serves as an intermediary between the presentation and data tiers.



MCQs + Subjective

CamScanner

Segments within layers

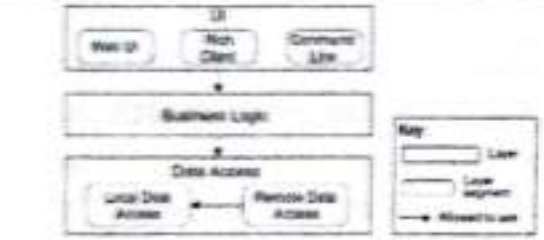


FIGURE 13.3 Layered design with segmented layers

MCQs

- Sometimes layers are divided into segments denoting a finer-grained decomposition of the modules.

Segments within layers

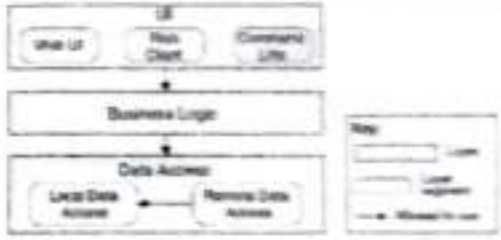


FIGURE 13.3 Layered design with segmented layers

MCQs

- Segments of the top layer are not allowed to use each other, but segments of the bottom layer are.

CamScanner

Broker Pattern

- **Solution:**
- The broker pattern separates users of services (clients) from providers of services (servers) by inserting an intermediary, called a broker.
- When a client needs a service, it queries a broker via a service interface.
- The broker then forwards the client's service request to a server, which processes the request.
- The service result is communicated from the server back to the broker, which then returns the result (and any exceptions) back to the requesting client.

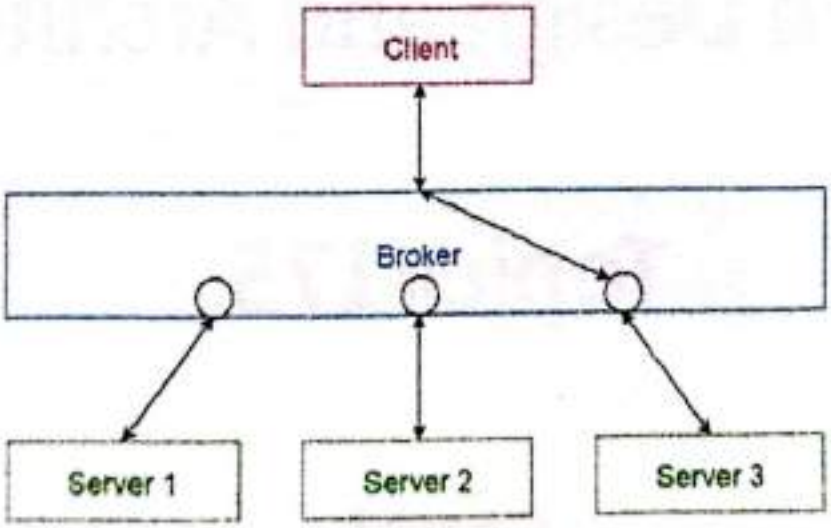
CamScanner

MCQs

- The first widely used implementation of the broker pattern was in the Common Object Request Broker Architecture (CORBA).
- Other common uses of this pattern are found in Enterprise Java Beans (EJB) and Microsoft's .NET platform
- Essentially any modern platform for distributed service providers and consumers implements some form of a broker.

MCQs

- The service-oriented architecture (SOA) approach depends crucially on brokers, most commonly in the form of an enterprise service bus.



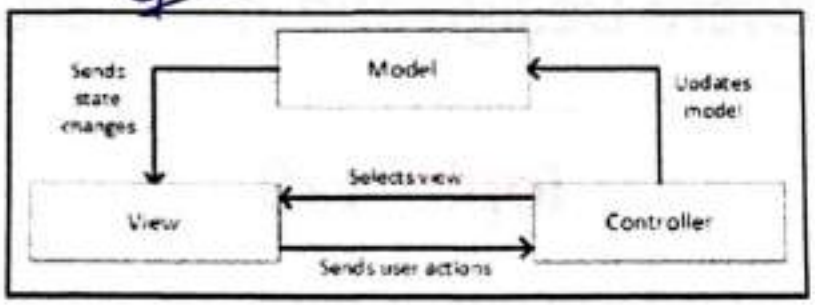
CamScanner

Software Design and Architecture

Topic#178
Component-and-Connector Patterns
MVC Pattern

Model-View-Controller (MVC)

- widely used for the UI of an application.
- It is particularly well suited to web applications, although it can also be used for other types of applications, such as desktop applications.
- The pattern provides a structure for building user interfaces and provides a separation of the different responsibilities involved.



- MVC is not appropriate for every situation.
  - The design and implementation of three distinct kinds of components, along with their various forms of interaction, may be costly, and this cost may not make sense for relatively simple user interfaces.

CamScanner

Model-View-Controller (MVC)

- A number of popular web and application development frameworks make use of this pattern.
  - A few examples include Ruby on Rails, ASP.NET MVC, and Spring MVC.

MVC

- the match between the abstractions of MVC and commercial user interface toolkits is not perfect.
  - The view and the controller split apart input and output, but these functions are often combined into individual widgets. This may result in a conceptual mismatch between the architecture and the user interface toolkit.

CamScanner

Pipe & filter Pattern

- **Solution:** The pattern of interaction in the pipe-and-filter pattern is characterized by successive transformations of streams of data.
- Data arrives at a filter's input port(s), is transformed, and then is passed via its output port(s) through a pipe to the next filter.
- A single filter can consume data from, or produce data to, one or more ports.

MCQs

Subj + MCQs

1236

Software Design and Architecture

✓

Topic#180

Component-and-Connector Patterns

Pipe-and-Filter Pattern – Strengths and Weaknesses

CamScanner



diagram

Pipe & filter Pattern

1237

→ Pipe & filter Pattern

MCQs.

- typically not a good choice for an interactive system, as it disallows cycles (which are important for user feedback).
- having large numbers of independent filters can add substantial amounts of computational overhead, because each filter runs as its own thread or process.

1240

CamScanner

Weakness

(3)

- may not be appropriate for long-running computations, without the addition of some form of checkpoint/restore functionality, as the failure of any filter (or pipe) can cause the entire pipeline to fail.

MCQs

→ • Pipes buffer data during communication. Because of this property, filters can execute asynchronously and concurrently. MCQs

CamScanner

- a filter typically does not know the identity of its upstream or downstream filters. For this reason, pipeline pipe-and-filter systems have the property that the overall computation can be treated as the functional composition of the computations of the filters, making it easier for the architect to reason about end-to-end behavior.

MCQs

- Data transformation systems are typically structured as pipes and filters, with each filter responsible for one part of the overall transformation of the input data.
- The independent processing at each step supports reuse, parallelization, and simplified reasoning about overall behavior.

Examples of Pipe-and-Filter Architecture

- systems built using UNIX pipes
- the request processing architecture of the Apache web server
- Yahoo! Pipes for processing RSS feeds
- many workflow engines
- many scientific computation systems that have to process and analyze large streams of captured data.

Software Design and Architecture

Topic#180

END!

CamScanner

Software Design and Architecture

Topic#181
Component-and-Connector Patterns
Client-Server Pattern

By: Miss Sabahat Saad
Youtube: Digital World VU
Whatsapp: 0326-1557791 (for paid)

- **Solution:**
  - Clients interact by requesting services of servers, which provide a set of services. Some components may act as both clients and servers.
  - There may be one central server or multiple distributed ones.

CamScanner

- Components
  - *clients* and *servers*
- Connectors
  - a data connector driven by a request/reply protocol used for invoking services.

Repeated

CamScanner

Strengths

Subjective

- The shared-data pattern is useful whenever various data items are persistent and have multiple accessors.
- Use of this pattern has the effect of decoupling the producer of the data from the consumers of the data; hence, this pattern supports modifiability, as the producers do not have direct knowledge of the consumers.

Strengths

- Consolidating the data in one or more locations and accessing it in a common fashion facilitates performance tuning.

Subjective + Objectives

Weaknesses

- the shared-data store may be a performance bottleneck.
  - For this reason, performance optimization has been a common theme in database research.
- The shared-data store is also potentially a single point of failure.

Weaknesses

- the producers and consumers of the shared data may be tightly coupled, through their knowledge of the structure of the shared data.

Software Design and Architecture

Topic#184

Allocation Patterns - Multi-tier Pattern

Multi-tier Pattern

MCQ

- The multi-tier pattern is a C&C pattern or an allocation pattern depending on the criteria used to define the tiers.

CamScanner

1272

Subjective

- A **monolithic architecture** is one in which a software application is designed to work as a single, self-contained unit.
- Applications that have this type of architecture are common.
- The components within a monolithic architecture are interconnected and interdependent, resulting in tightly coupled code.

1275

Benefits

- Applications with a monolithic architecture typically have better performance.
- Small applications that have this type of architecture are easier to deploy because of the simplicity of the high-level architecture.

CamScanner

1279

1280

(3) • In spite of the tightly coupled logic, monolithic applications can be easier to test and debug because they are simpler, with fewer separate components to consider.

Imp 4 Subjective + McQs

1282

(4) • Monolithic applications are typically easy to scale because all it takes is to run multiple instances of the same application.
  - However, different application components have different scaling needs and we cannot scale the components independently with a monolithic architecture.
  - We are limited to adding more instances of the entire application in order to scale.

1283

Drawbacks

(1) • Monolithic applications greatly inhibit the agility of the organization as it becomes difficult to make changes to the software.

(2) • continuous deployment is difficult to achieve.
  - Even if a change is made to only one component of a monolithic application, the entire software system will need to be deployed.

1285

(3) • Organizations are required to devote more resources, such as time and testers, to deploy a new version of a monolithic application.

CamScanner

(4)

- Low maintainability
  - Tightly coupled components make it more difficult to make changes because a change in one part of the application is more likely to affect other parts of the application.
  - Difficult to understand

CamScanner

(5)

- It also takes longer for such applications to start up, lowering the productivity of the team during development.

Contact us at: 0326-1557791

for Paid Projects (CS619) & LMS Handling

Miss Sabahat Saad

CamScanner

- Why Web Application
  - Almost 23 million software developers (2018) MCQs
  - Over 15 million of these actively develop software for the web or related technologies

Design Considerations MCQs + Subj

- **When designing a Web application, the goals of a software architect are to minimize the complexity by separating tasks into different areas of concern while designing a secure, high performance application.**

CamScanner

Web Application Overview

- The core of a Web application is its server-side logic.
- The Web application layer itself can be comprised of many distinct layers.
- The typical example is a three-layered architecture comprised of presentation, business, and data layers.

CamScanner

- **Partition your application logically**
  - Use layering to partition your application logically into presentation, business, and data access layers.
  - helps to create maintainable code
  - allows to monitor and optimize the performance of each layer separately.
  - offers more choices for scaling your application.

CamScanner

1297

Subjective

1298

2 • **Use abstraction to implement loose coupling between layers.**
  - This can be accomplished by defining interface components, such as a façade (or interface types/abstract base classes) with well-known inputs and outputs that translates requests into a format understood by components within the layer.

1300

3 • **Understand how components will communicate with each other**
  - This requires an understanding of the deployment scenarios your application must support.
  - You must determine if communication across physical boundaries or process boundaries should be supported, or if all components will run within the same process.

1301

Software Design and Architecture

Topic#189
Web Application Design Considerations
Part II - Performance

Points to remember

1303

1 • **Reduce round trips.**
  - When designing a Web application, consider using techniques such as caching and output buffering to reduce round trips between the browser and the Web server, and between the Web server and downstream servers.

CamScanner

- **Consider using caching**
  - A well-designed caching strategy is probably the single most important performance-related design consideration.

CamScanner

Performance

(3) • **Avoid blocking during long-running tasks**
  - If you have long-running or blocking operations, consider using an asynchronous approach to allow the Web server to process other incoming requests.

Security

(1) • **Consider using logging and instrumentation**
  - You should audit and log activities across the layers and tiers of your application.
  - These logs can be used to detect suspicious activity, which frequently provides early indications of an attack on the system.

Security

(4) • **Design your Web application to run using a least-privileged account**
  - If an attacker manages to take control of a process, the process identity should have restricted access to the file system and other system resources in order to limit the possible damage.

Subjective

Security

(2) **•Consider authenticating users across trust boundaries**

- You should design your application to authenticate users whenever they cross a trust boundary; for example, when accessing a remote business layer from your presentation layer.

Security

(3) **•Do not pass sensitive data in plain text across the network.**

- Whenever you need to pass sensitive data such as a password or authentication cookie across the network, consider encrypting and signing the data or using Secure Sockets Layer (SSL) encryption.

CamScanner

Software Design and Architecture

IMP.

Topic#192

Web Application Frame - Authentication

1) Authentication
2) Authorization
3) Caching
4) Exception Management
(9) Presentation Entity
(11) Session Management
(5) Logging & Instrumentation
(6) Navigation
(7) Page layout
(8) Page Rendering
(10) Request Processing
(12) Validation

Guidelines

CamScanner

1315

Authentication

MCQs

- Designing an effective authentication strategy is important for the security and reliability of your application.
  - Improper or weak authorization can leave your application vulnerable to spoofing attacks, dictionary attacks, session hijacking, and other types of attack.

1318

1316

Q: Write down common mistakes in authentication process?

Common Mistakes

1. Lack of authentication across trust boundaries
2. Storing passwords in a database as plain text
3. Designing custom authentication mechanism instead of using built-in capabilities.

1319

(a)

(i)

G

- Ide
- This
- Use wh
- Enf acc
- En pa po

CamScanner

- Authorization determines the tasks that an authenticated identity can perform, and identifies the resources that can be accessed. Designing an effective authorization strategy is important for the security and reliability of your application.
- Improper or weak authorization leads to information disclosure, data tampering, and elevation of privileges.
- Defense in depth is the key security principle to apply to your application's authorization strategy.

CamScanner

Q: subjective + MCQs

Common Mistakes

- Lack of authorization across trust boundaries
- Incorrect role granularity
- Using impersonation and delegation when not required

CamScanner

Define Caching subj + obj

- Caching improves the performance and responsiveness of your application.
- However, incorrect caching choices and poor caching design can degrade performance, security and responsiveness.
- You should use caching to optimize reference data lookups, avoid network round trips, and avoid unnecessary and duplicate processing.
- To implement caching, you must first decide when to load data into the cache.
- Try to load cache data asynchronously or by using a batch process to avoid client delays.

MCQs

MCQs

CamScanner

Common Mistakes

- Caching volatile data
- Not considering caching page output
- Caching sensitive data
- Failing to cache data in a ready-to-use format

Guidelines

- Avoid caching volatile data.
- Use output caching to cache pages that are relatively static.
- Consider using partial page caching through user controls for static data in your pages.
- Pool shared resources that are expensive, such as network connections, instead of caching them.
- Cache data in a ready-to-use format.

CamScanner

Software Design and Architecture

Topic#195

Web Application Frame – Exception Management

- Designing an effective exception management strategy is important for the security and reliability of your application.
- Correct exception handling in your Web pages prevents sensitive exception details from being revealed to the user, improves application robustness, and helps to avoid leaving your application in an inconsistent state in the event of an error.

CamScanner

Guidelines

- Do not use exceptions to control the logical flow of your application.
- Do not catch exceptions unless you must handle them, you need to strip sensitive information, or you need to add additional information to the exception.
- Design a global error handler to catch unhandled exceptions.

Software Design and Architecture

Topic#196
Web Application Frame – Logging and Instrumentation

CamScanner

Guidelines

- Consider auditing for user management events.
- Consider auditing for unusual activities.
- Consider auditing for business-critical operations.
- Create secure log file management policies, such as restricting the access to log files, allowing only write access to users, etc.
- Do not store sensitive information in the log or audit files.

CamScanner

Software Design and Architecture

Topic#197
Web Application Frame – Navigation

- Design your navigation strategy in a way that separates it from the processing logic.
- Your strategy should allow users to navigate easily through your screens or pages.
- Designing a consistent navigation structure for your application will help to minimize user confusion as well as reduce the apparent complexity of the application.

Guidelines

- Design a site map to help users find pages on the site, and to allow search engines to crawl the site if desired.
- Consider using wizards to implement navigation between forms in a predictable way.

Guidelines

- Consider using visual elements such as embedded links, navigation menus, and breadcrumb navigation in the UI to help users understand where they are, what is available on the site, and how to navigate the site quickly.

CamScanner

Guidelines

- Use well-known design patterns, such as Model-View-Controller (MVC), to decouple UI processing from output rendering.
- Consider encapsulating navigation in a master page so that it is consistent across pages.

CamScanner

Software Design and Architecture

Topic#198
Web Application Frame – Page Layout

- Design your application so that the page layout can be separated from the specific UI components and UI processing.
- When choosing a layout strategy, consider whether designers or developers will be building the layout.
- If designers will be building the layout, choose a layout approach that does not require coding or the use of development-focused tools.

CamScanner

Imp 4 Subj + Obj

Guidelines

MCQs

- Use Cascading Style Sheets (CSS) for layout whenever possible. MCQs
- Use table-based layout when you need to support a grid layout, but remember that table-based layout can be slow to render, does not have full cross-browser support, and there may be issues with complex layout.

CamScanner

Guidelines

- Consider data-binding options.
  - For example, you can bind custom objects or datasets to controls.
- Consider using Asynchronous JavaScript and XML (AJAX) for an improved user experience and better responsiveness.
- Consider using data-paging techniques for large amounts of data to minimize scalability issues.

Software Design and Architecture

Topic#200

Web Application Frame – Presentation Entity

CamScanner

Sujit Oli

By: Sabahat Saad
For paid project contact
0326-1557791

- Presentation entities store the data that you will use to manage the views in your presentation layer.
- Presentation entities are not always necessary.
- Consider using presentation entities only if the datasets are sufficiently large or complex that they must be stored separately from the UI controls.
- Design or choose appropriate presentation entities that you can easily bind to UI controls.)

CamScanner

Guidelines

- Determine if you need presentation entities.
  - Typically, you might need presentation entities if the data or data format to be displayed is specific to the presentation layer. MCQs
- Consider the serialization requirements for your presentation entities, if they are to be passed across the network or stored on the disk.

Software Design and Architecture

Topic#201

Web Application Frame – Request Processing

CamScanner

recover from a crash, make sure after recovery that the user interface is in a consistent state.

1361

1362

- When designing a request-processing strategy, you should ensure separation of concerns by implementing the request-processing logic separately from the UI.

1364

Guidelines

- Consider centralizing the common pre-processing and post-processing steps of Web page requests to promote logic reuse across pages.
- Consider dividing UI processing into three distinct roles—model, view, and controller/presenter—by using the Model-View-Controller (MVC) or Model-View-Presenter (MVP) pattern.

1365

CamScanner

Guidelines

- If you are designing views for handling large amounts of data, consider giving access to the model from the view by using the Supervising Controller pattern, which is a form of the MVP pattern.

MVPs

CamScanner

Software Design and Architecture

Topic#202
Web Application Frame – Session Management

CamScanner

- When designing a Web application, an efficient and secure session-management strategy is important for performance, security and reliability.
- You must consider session-management factors such as what to store, where to store it, and how long information will be kept.

MCQs

CamScanner

Software Design and Architecture

Topic#203
Web Application Frame – Validation

Q: Validation? Imp + Obj

- Designing an effective validation solution is important for the security and reliability of your application.
- Improper or weak authorization can leave your application vulnerable to cross-site scripting attacks, SQL injection attacks, buffer overflows, and other types of input attack.

CamScanner

Guidelines

MCQ

- Use Web user controls if you need to reuse UI fragments on several pages, or if you want to cache a specific parts of the page.

CamScanner

Guidelines

- Design your business layer to be stateless.
  - This helps to reduce resource contention and increase performance. MCQs
- Use a message-based interface for the business layer.
  - This works well with a stateless Web application business layer.

Software Design and Architecture

Topic#206
Web Application Frame
Data Layer Considerations

CamScanner

- Design a data layer for your Web application to abstract the logic necessary to access the database.
- Using a separate data layer makes the application easier to configure and maintain.
- The data layer may also need to access external services using service agents.

CamScanner

Guidelines

- Design a separate data layer to hide the details of the database from other layers of the application.
- Design entity objects to interact with other layers, and to pass the data between them.
- Design to take advantage of connection pooling to minimize the number of open connections.

CamScanner

Guidelines

MCQs

- If your business layer is on a remote tier, design coarse-grained service methods to minimize the number of client-server interactions, and to provide loose coupling.
- Design the services without assuming a specific client type.
- Design the services to be idempotent, assuming that the same message request may arrive multiple times.

Subj + Obj

Testability is a measure of how well your system or components allow you to create test criteria and execute tests to determine if the criteria are met.

- You should consider testability when designing your architecture because it makes it easier to diagnose problems earlier and reduce maintenance cost.
- To improve the testability of your application, you can use logging events (1), provide monitoring (2) resources, and implement test interfaces. (3)

CamScanner

- When deploying a Web application, you should take into account how layer and component location will affect the performance, scalability, and security of the application.
- You might also need to consider design trade-offs.
- Use either a distributed or a non-distributed deployment approach, depending on the business requirements and infrastructure constraints.

MCQs

Guidelines

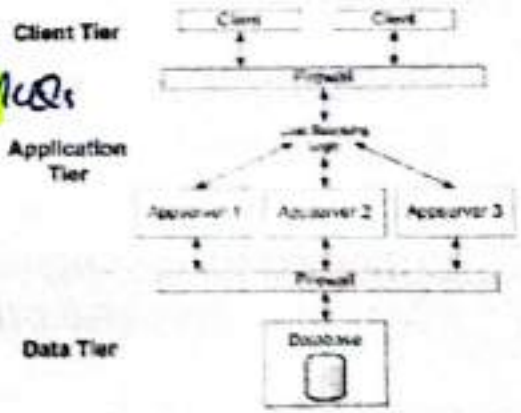
- Consider using non-distributed deployment to maximize performance.
- Consider using distributed deployment to achieve better scalability and to allow each layer to be secured separately.

Software Design and Architecture

Topic#212
Web Application Frame
Load Balancing

- When you deploy your Web application on multiple servers, you can use load balancing to distribute requests so that they are handled by different Web servers.
- This helps to maximize response times, resource utilization, and throughput.

MCQs



CamScanner

Guidelines

MCOs

- Avoid server affinity when designing scalable Web applications.
  - Server affinity occurs (1) when all requests from a particular client must be handled by the same server.
  - It usually occurs (2) when you use locally updatable caches, or in-process or local session state stores.

CamScanner

END!

1424

Architecture in the cloud

Cloud Computing

Short + MCQs
V. Imp

- Cloud computing is the on-demand availability of computer system resources, especially data storage and computing power, without direct active management by the user.

Digital World Vu

By- Sabahat Saeed
For Paid Projects CS619/CS519 (03261557791)

Cloud Computing

- The term is generally used to describe data centers available to many users over the Internet.
- Cloud computing relies on sharing of resources to achieve coherence and economies of scale.

MCQs

CamScanner

Cloud Computing

MCQs

- It provides an elastic set of resources through the use of virtual machines, MCQs virtual networks, and virtual file systems. MCQs
- The cloud has become a viable MQs alternative for the hosting of data centers primarily for economic reasons.

CamScanner

Basic Cloud Definitions

- The essential characteristics of cloud computing
  - based on definitions provided by the U.S. National Institute of Standards and Technology, or NIST

means

CamScanner

Basic Cloud Definitions

Define Cloud?

- *On-demand self-service.*
  - A resource consumer can unilaterally provision computing services, such as server time and network storage, as needed automatically without requiring human interaction with each service's provider.

1432

Basic Cloud Definitions

- *On-demand self-service.*
  - This is sometimes called empowerment of end users of computing resources.
  - Examples of resources include storage, processing, memory, network bandwidth, and virtual machines.

1433

- *Resource pooling*
  - The cloud provider's computing resources are pooled.
  - In this way they can efficiently serve multiple consumers.
  - The provider can dynamically assign physical and virtual resources to consumers, according to their instantaneous demands.

1435

- *Location independence.*
  - The location independence provided by ubiquitous network access is generally a good thing.

CamScanner

short+MCQs

- *Ubiquitous network access.*
  - Cloud services and resources are available over the network and accessed through standard networkin mechanisms that promote use by a heterogeneous collection of clients.
  - This capability is independent of location and device;
    - all you need is a client and the Internet.

MCQs

MCQs

- *Rapid elasticity.* short+mcqs
  - Due to resource pooling, it is easy for capabilities to be rapidly and elastically provisioned, in some cases automatically, to quickly scale out or in.
  - To the consumer, the capabilities available for provisioning often appear to be virtually unlimited.

MCQs

- *Multi-tenancy.*
- Multi-tenancy is the use of a single application that is responsible for supporting distinct classes or users.
- Each class or user has its own set of data and access rights, and different users or classes of users are kept distinct by the application.

CamScanner

diff b/w (SaaS) & (PaaS) & (IaaS)?

Software as a Service (SaaS) Imp

- The consumer in this case is an end user.
- The consumer uses applications that happen to be running on a cloud.
- The applications can be as varied as email, calendars, video streaming, and real-time collaboration.

1441

Platform as a Service (PaaS)

- The consumer in this case is a developer or system administrator.
- The platform provides a variety of services that the consumer may choose to use.
- These services can include various database options, load-balancing options, availability options, and development environments.

Infrastructure as a Service (IaaS)

- The consumer in this case is a developer or system administrator.
- The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications.

options, and development environments.

1442

and applications

Imp

1443

Q: Diff b/w private & Public Clouds.

Deployment Models

- *Private cloud.*
  - The cloud infrastructure is owned solely by a single organization and operated solely for applications owned by that organization.
  - The primary purpose of the organization is not the selling of cloud services.

1445

Deployment Models

- *Public cloud.*
  - The cloud infrastructure is made available to the general public or a large industry group and is owned by an organization selling cloud services.

1446

re

1447

Software Design and Architecture

Topic#216
Cloud Computing – Multitenancy

1448

Q: Define

- Multi-tenancy applications are architected explicitly to have a single application that supports distinct sets of users.
- The economic benefit of multi-tenancy is based on the reduction in costs for application update and management.

1449

CamScanner

Security

- Applications in the cloud are accessed over the Internet using standard Internet protocols.
- The security and privacy issues deriving from the use of the Internet are substantial but no different from the security issues faced by applications not hosted in the cloud.

Security

- The one significant security element introduced by the cloud is multi-tenancy.
  - Multi-tenancy means that your application is utilizing a virtual machine on a physical computer that is hosting multiple virtual machines.
  - If one of the other tenants on your machine is malicious, what damage can they do to you?

Performance

- Regardless of whether the provider automatically allocates additional resources, the application should be self-aware of both its current resource usage and its projected resource usage.

Availability

- The cloud is assumed to be always available. But everything can fail

CamScanner

Performance

- One virtue of the cloud is that it provides an elastic host.
- Elasticity means that additional resources can be acquired as needed.

1461

Availability

- The service-level agreement of some cloud service provider provides a 99.95 percent guarantee of service.
- There are two ways of looking at that number:
  - That is a high number. You as an architect do not need to worry about failure.
  - That number indicates that the service may be unavailable for .05 percent of the time. You as an architect need to plan for that .05 percent.

1464

Software Design and Architecture

Fakhar Lodhi

Topics Covered: 218-221

Service-Oriented Architecture

CamScanner

Imp

- **Service-oriented architecture (SOA)** *is an* architectural pattern for developing software systems by creating loosely coupled, interoperable services that work together to automate business processes.

Definition

CamScanner

MCQ + Subj

(4) • A service is a part of a software application that performs a specific task, providing functionality to other parts of the same software application or to other software applications.

• Some examples of service consumers include web applications, mobile applications, desktop applications, and other services.

1470

MCQs

(6) • Each service in a SOA encapsulates a certain piece of logic.

(7) • This logic may be responsible for a very specific task, a business process, or a sub-process.

(8) • Services can vary in size and one service can be composed of multiple other services to accomplish its task.

1473

MCQs

(3) SOA achieves a SoC, which is a design principle that separates a software system into parts, with each part addressing a distinct concern.

(4) • A key aspect of SOA is that it decomposes application logic into smaller units that can be reused and distributed.

1471

(5) • By decomposing a large problem into smaller, more manageable concerns satisfied by services, complexity is reduced and the quality of the software is improved.

1472

CamScanner

Software Design and Architecture

Topic#220

Benefits of Service-Oriented Architecture

Notes By : Sabahat Saad
For Paid Projects & LMS Handling
Contact at : 0326-1557791
UTube : Digital World VU

CamScanner

services to accomplish its task.

1. • Increases alignment between business and technology
   - Business logic, in the form of business entities and business processes, exists in the form of physical services with an SOA.
2. • Promotes federation within an organization
   - Organizations have the flexibility to choose whether they want to replace certain systems, allowing them to use a phased approach to migration.

CamScanner

3. Allows for vendor diversity
4. Increases intrinsic interoperability
   - It allows for the sharing of data and the reuse of logic. Different services can be assembled together to help automate a variety of business processes.
5. Works well with agile development methodologies
   - each task can be made to be manageable in size and more easily understood.

MCQS

CamScanner

Short + MCQs

- The **microservice architecture (MSA)** pattern builds software applications using small, autonomous, independently versioned, self-contained services.
- These services use well-defined interfaces and communicate with each other over standard, lightweight protocols.

CamScanner

2-

(3) •Interaction with a microservice takes place through a well-defined interface.

MCQs

(4) •A microservice should be a *black box* to the consumers of the service, hiding its implementation and complexity.

MCQs

CamScanner

- Each microservice focuses on doing one thing well and they can work together with other microservices in order to accomplish tasks that are more complex.

- A microservice architecture is particularly well-suited for large and/or complex software systems.
- In contrast with the monolithic architecture, applications built using a microservice architecture handle complexity by splitting the application into smaller services that are easier to manage.

Software Design and Architecture

Topic#223

Characteristics of microservice architecture

Imp Subjective

1) Small, focused services
2) Well-defined service interfaces
3) Autonomous and independently deployable services
4) Independent data storage
5) Communicating with lightweight protocols
6) Better fault isolation

- Incoming requests are commonly handled by an API gateway MCQs which serves as the entry point to the system. MCQs
- It is an HTTP server that takes requests from clients and routes them to the appropriate microservice through its routing configuration.

Software Design and Architecture

Topic#223

END!

CamScanner

Software Design and Architecture

Topic#224
Designing Polyglot Services

CamScanner

- Monolithic applications focus on using a particular programming language and technology stack.
- Because that type of application is written as a single unit, it is more difficult to take advantage of different types of technology.

1504

CamScanner

- Polyglot
  - a person who knows and is able to use several languages

CamScanner

Subjective

+ MCQs.

- Two of the concepts related to polyglot microservices are:
- polyglot programming
  - With **polyglot programming**, a single application uses multiple programming languages in its implementation.
- polyglot persistence
  - multiple persistence options are used within a single application

CamScanner

Software Design and Architecture

Topic#225
Stateless versus Stateful Microservices

CamScanner

- Stateful applications store state from client requests on the server itself and use that state to process further requests.
  - When a user sends a login request, it enables login to be true and user authenticated now, and on the second request, the user sees the dashboard.

- Stateful applications don't need to make a call to DB second time as session info stored on the server itself, hence it is faster.

- Stateless applications don't store any state on the server.
- Any data that flows via a stateless service is typically transitory and the state is stored only in a separate back-end service like a database.
  - Any associated storage is typically transitory.
  - If the container restarts for instance, anything stored is lost.

- Each microservice can either be *stateless* or *stateful*.

CamScanner

Problems with Stateful Apps ?

- Load balancing and scalability

Subjective diff b/w ?,

Stateless	Stateful
Requests are self-contained	Shorter messages
Better fault tolerance	Better performance
No space reserved for file descriptor tables	File locking possible
No limit on open files	Read ahead possible
No problem if client crashes	

CamScanner

- Decomposing a complex system into the right set of microservices can be difficult.
- It requires a knowledge of the domain and can be somewhat of an art.

1523

- The use of multiple databases is another challenge when using a microservice architecture.
- It is common for a business transaction to update multiple entities, which will require the use of multiple microservices.
- With each one having its own database, this means that updates must take place in multiple databases.

1526

CamScanner